

浙师大校园”微交通”系统的综合治理与优化设计

摘 要

针对浙师大扩招后突出的校园拥堵与微交通安全隐患，本文基于 84 节点 310 边的路网拓扑，建立冲突风险识别、安全承载力评估与公交选址调度模型，提出分阶段综合治理方案。

针对问题一，以引力模型生成三时段 OD 需求，建立 BPR 用户均衡模型并构造人车冲突风险指标 S 。构建 MinMax V/C 增量分流算法，将各 OD 需求按 60 批次动态释放并更新阻抗，使瓶颈路段饱和度从 1.249 降至 0.255（降幅 79.6%），全路网过饱和路段清零，同时输出各 OD 对的最优分流路径供导流标识设置使用。

针对问题二，构建“静态停车—动态道路—消防折减”承载力校核模型。测算得出全校电动车安全承载力为 9414 辆，判定 16 号楼、25 号楼等高峰超载区停车空间错配是主因，需实施错峰分布与总量控制。

针对问题三，建立 MCLP 最大覆盖选址与潮汐调度模型，优选 5 个核心站点规划“桃园—桂苑”公交潮汐线。以系统广义成本最小化配置 3 辆接驳车，使校园电动车出行需求有效削减 10.8%，冲突风险下降 3.8%。

针对问题四，基于“安全优先、总量控制、核心限停、外围分流”等维度设计分阶段治理方案，形成面向管理者的决策建议。本文各项模型数据逻辑严密衔接，框架可推广至同类高校或封闭园区。

关键词：BPR-UE 交通分配 环境承载力评估 MCLP 最大覆盖选址 微公交潮汐调度

一、 交通流分配与冲突风险评价

1.1 问题重述

校园高峰期教学区、宿舍区、食堂间出行集中，主干道上人车混行明显，易出现拥堵和安全风险。问题一要求识别核心路段和瓶颈位置，设计单行线、分时限行、人车隔离等交通组织方案。重点给出 16 号楼、25 号楼到餐厅和宿舍的分流路径，并量化方案对通行效率的改善。

1.2 问题分析

问题一主要用校园有向图刻画路网结构，用引力模型生成早、中、晚三时段 OD 需求，用 BPR 函数描述路段拥堵，用用户均衡模型分配交通流。评价时结合总行程时间、道路饱和度和人车冲突风险指标，最后用方案比选确定最优分流组织方案。

1.3 模型假设

作如下假设：

1. 校园道路、建筑出入口和主要交叉口可抽象为有向加权图，路段长度和宽度在研究时段内保持不变；
2. 高峰期 OD 需求主要由宿舍、教学楼、食堂和商业街之间的潮汐出行产生，可用建筑容量和网络距离估计；
3. 步行、电动车和机动车在同一路段上存在混行干扰，可通过等效 PCU 系数折算为统一流量；
4. 出行者倾向于选择当前阻抗较小的路径，整体交通分配可用用户均衡思想近似；
5. 人车隔离措施可显著降低不同交通方式之间的空间交织风险，但可能带来少量绕行成本。

1.4 符号说明

表 1-1: 问题一主要符号说明

符号	说明	单位
$G = (V, E)$	校园有向道路网络	—
l_e	路段 e 的长度	m
w_e	路段 e 的宽度	m
C_e	路段 e 的通行能力	pcu/h
t_e^0	路段自由流通行时间	s
X_e^h	时段 h 路段 e 的等效 PCU 流量	pcu/h
$q_{ij}^{m,h}$	时段 h 中方式 m 的 OD 需求	人次
TTT	全路网总行程时间	人·s
S	全路网人车冲突风险指标	—
μ_e^h	路段饱和度，即 V/C	—

1.5 模型的建立与求解

1.5.1 校园路网与 OD 需求估计

将校园道路系统抽象为有向加权图

$$G = (V, E), \quad (1-1)$$

其中 V 为节点集合，包括教学楼、宿舍区、食堂、商业街及道路交叉口； E 为有向路段集合。每条路段 $e \in E$ 具有长度 l_e 、宽度 w_e 、通行能力 C_e 和自由流速度 v_e^0 。扩展版校园路网包含 84 个节点、310 条有向边，覆盖宿舍区、教学区、食堂、商业街和主要交叉口，其拓扑结构如图1-1所示。



图 1-1: 浙师大校园交通网络图，含 84 个节点、310 条有向边

在本文路网模型中，建筑节点（宿舍、教学楼、食堂、商业街）并非直接坐落在道路上，而是作为交通发生/吸引的形心节点独立存在。每栋建筑通过 2 ~ 5 条虚拟连接线挂载至周边最近的道路交叉口节点，形成辐条式接入结构。由于缺少逐 OD 实测数据，本文采用引力模型估计高峰期出行分布^[1]。时段 h 下，从建筑 i 到建筑 j 的出行需求为

$$T_{ij}^h = O_i^h \cdot \frac{D_j (d_{ij})^{-\gamma}}{\sum_k D_k (d_{ik})^{-\gamma}}, \quad (1-2)$$

其中 O_i^h 为出发建筑在时段 h 的出行产生量， D_j 为到达建筑吸引力， d_{ij} 为网络最短路径距离；考虑到校园封闭路网内师生对中长距离（如宿舍到食堂）出行敏感度的适中特性，距离衰减系数取 $\gamma = 1.5$ 。

根据公开数据显示，全校本科生共计 28,000 左右，建筑容量与时段负荷系数作为 OD 生成参数（宿舍早 0.90/教学楼午 0.93/教学楼晚 0.86）。

在得到 OD 总量后，按出行距离划分交通方式：短距离以步行为主，中长距离以电动车为主，远距离保留少量机动车需求。三时段 OD 估计结果见表1-2。

表 1-2: 三时段 OD 需求估计结果

时段	主要方向	OD 总量/人次	电动车人次
早高峰	宿舍区 → 教学区	11,340	4,146
午高峰	教学区 → 食堂/商业街	12,000	4,393
晚高峰	教学区 → 宿舍/食堂	12,900	4,542

1.5.2 多方式用户均衡分配模型

为统一刻画步行、电动车和机动车的混行影响，将三类方式转换为等效 PCU 流量：

$$X_e^h = \sum_{m \in M} \xi_m x_e^{m,h}, \quad M = \{p, b, c\}, \quad (1-3)$$

其中 p, b, c 分别表示步行、电动车和机动车， $x_e^{m,h}$ 为方式 m 在路段 e 、时段 h 的流量， ξ_m 为等效换算系数。路段阻抗采用 BPR 函数：

$$t_e^h = t_e^0 \left[1 + \alpha \left(\frac{X_e^h}{C_e} \right)^\beta \right], \quad (1-4)$$

其中 $t_e^0 = l_e/v_e^0$ 。在缺乏大规模实测校准数据的情况下，阻抗参数参考美国联邦公路局 (FHWA) 标准 BPR 函数推荐值^[2]，取 $\alpha = 0.15$ ， $\beta = 4.0$ ，以较好地刻画流量逼近通行能力时拥堵的急剧恶化现象。

用户均衡假设为：同一 OD 对、同一交通方式下，被选择路径的出行阻抗相等且不大于未被选择路径^[3]。对应的 Beckmann 等价规划为

$$\begin{aligned} \min \quad & Z = \sum_{e \in E} \int_0^{X_e^h} t_e(w) dw, \\ \text{s.t.} \quad & \sum_{k \in K_{ij}} f_{k,ij}^{m,h} = q_{ij}^{m,h}, \quad \forall (i,j), m, h, \\ & x_e^{m,h} = \sum_{(i,j)} \sum_{k \in K_{ij}} \delta_{e,k}^{ij} f_{k,ij}^{m,h}, \quad \forall e, m, h, \\ & f_{k,ij}^{m,h} \geq 0. \end{aligned} \quad (1-5)$$

采用 Frank-Wolfe 算法求解^[4]。每轮迭代执行全有全无分配获得下降方向，一维线搜索确定步长，至收敛条件满足。

1.5.3 微观精细化路径分流模型

在上述宏观用户均衡 (UE) 分配模型的基础上，为进一步解决特定核心楼宇（如 16 号楼、25 号楼）出入导致的微观流线交织问题，本文在此建立 MinMax V/C 增量分配模型。模型核心思想是将各时段 OD 需求按 $B = 60$ 个小批次等量切分，利用动态阻抗将车流“自适应铺摊”到低负荷路径上。

具体地，微观路段 e 在第 b 批分配时的动态阻抗定义为

$$t_e^b(m) = t_e^0(m) \cdot \left[1 + \lambda \cdot \left(\frac{X_e^{b-1}}{C_e} \right)^{\beta_p} + \eta \cdot \max\left(0, \frac{X_e^{b-1}}{C_e} - 0.75\right)^2 + \gamma \cdot \max\left(0, \frac{8 - w_e}{8}\right) \right], \quad (1-6)$$

其中 $\lambda = 65$ 、 $\beta_p = 4$ 控制拥堵惩罚强度（经网格测试标定，既能消除路段“拥死”也能避

免过度绕行)； $\eta = 140$ 作为高敏感系数，对 $V/C > 0.75$ 的高危过饱和路段施加额外超容惩罚； $\gamma = 0.4$ 对窄路（宽度 $< 8\text{m}$ ）施加通行折扣以模拟窄路在混行条件下的通行能力剥减。对于特定楼宇的定向疏导，三方式（步行、电动车、机动车）将分别依此阻抗独立进行增量 Dijkstra 最短路径搜索。

1.5.4 评价指标与现状基准分析

结合上述模型算法，首先制定量化评价指标。模型评价指标包括总行程时间、冲突风险和道路饱和度。总行程时间定义为

$$TTT = \sum_{h \in \mathcal{H}} \sum_{e \in E} \sum_{m \in M} x_e^{m,h} t_e^h. \quad (1-7)$$

人车冲突风险定义为不同方式流量乘积的加权和：

$$S_e^h = \omega_{pb} x_e^{p,h} x_e^{b,h} + \omega_{pc} x_e^{p,h} x_e^{c,h} + \omega_{bc} x_e^{b,h} x_e^{c,h}, \quad (1-8)$$

其中风险权重系数根据不同交通工具的质量差和速度差决定：机动车与行人混行的伤害严重程度最大（设定 $\omega_{pc} = 1.5$ ），机动车与电动车次之（设定 $\omega_{bc} = 0.8$ ），电动车与行人的相对质量差小，伤害风险相对较低（设定 $\omega_{pb} = 0.5$ ）。全路网风险为 $S = \sum_{h,e} S_e^h$ 。道路饱和度为 $\mu_e^h = X_e^h / C_e$ ，用于识别宏观层面的瓶颈路段。

在不采取治理措施的基准情景下，三个时段分别进行用户均衡分配，结果如表1-3所示。午高峰 TTT 和 S 均为三时段最高，原因是 12 栋教学楼同时向食堂和商业街方向集中释放出行需求，多方向流线在网络中叠加，冲突风险显著高于早、晚的单向通勤流。

表 1-3: 现状情景三时段 UE 分配结果

时段	TTT	冲突风险 S	核心瓶颈路段
早高峰	2,557,530	7,651,286	$N4 \rightarrow N14$, $V/C=1.249$
午高峰	2,980,097	11,419,251	$N16 \rightarrow N30$, $V/C=1.410$
晚高峰	2,490,034	7,636,591	$N14 \rightarrow N4$, $V/C=1.251$
合计	8,027,661	26,707,128	—

1.5.5 宏观交通组织方案设计与安全评价

针对现状瓶颈和冲突风险分布，从路网结构层面设计三类方案进行比选。方案 A 在早高峰关键路段设置单行线（ $N30 \rightarrow N16$ 、 $N4 \rightarrow N14$ 、 $N49 \rightarrow N48$ ）；方案 B 在午高峰对 $N16 \rightarrow N30$ 、 $N48 \rightarrow N49$ 等 11 条高 V/C 路段降容 60% 并削减机动车需求至 30%；方案 C 为方案 C，将方案 A、方案 B 与高风险路段人车隔离合并，对冲突风险最高的 18 条路段设置物理隔离。

三类方案的效果比较见表1-4（ ΔTTT 负值表示总行程时间增加， ΔS 正值表示安全水平改善）：

表 1-4: 三类治理方案效果对比

方案	TTT	ΔTTT	冲突风险 S	ΔS
基准现状	7,949,785	—	28,963,236	—
方案 A: 单行线	8,120,949	-1.1%	27,403,631	+5.4%
方案 B: 限行 + 削减	8,118,808	-1.0%	24,903,939	+14.0%
方案 C	8,223,148	-2.2%	22,030,958	+25.3%

方案 C 的 $\Delta S = 25.3\%$ ，高于方案 A (5.4%) 和方案 B (14.0%)。 $\Delta TTT = -2.2\%$ ，增量来自单行线和限行造成的绕行。

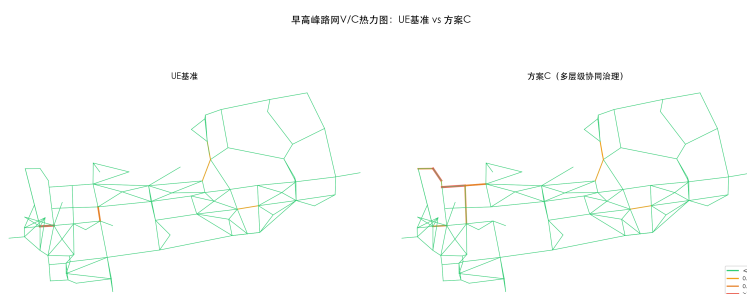


图 1-2: 早高峰路网 V/C 热力图 (左: UE 基准, 右: 方案 C 改造后)

图1-2显示, 方案 C 将 N4→N14 路段 V/C 从 1.249 降至 1.053, 但造成负面转移: N8→N9 与 N10→N11 路段骤升至 1.053, 全网过饱和路段从 1 条增至 2 条。方案 C 以降低冲突风险换取路网效率, 但绕行诱发了新的拥堵点, 难以兼顾安全与效率。

1.5.6 微观分流效果评估与最优路径生成

上述宏观方案从路网结构层面降低了冲突风险, 但存在两个局限: 一是物理改造 (如单行线) 会强制绕行, 可能诱发新的拥堵点; 二是 UE 分配中所有出行者仍选择单一最短路径, 导致瓶颈路段流量集中。为此, 本节在不改动路网结构的条件下, 建立 MinMax V/C 增量分配模型, 通过动态路径引导实现车流的”自适应铺摊”。

模型将各时段 OD 需求按 $B = 60$ 批次依次释放, 每批次根据当前饱和度更新阻抗, 使后续车流自动避开已拥堵路段。三时段最大 V/C 变化如表1-5所示:

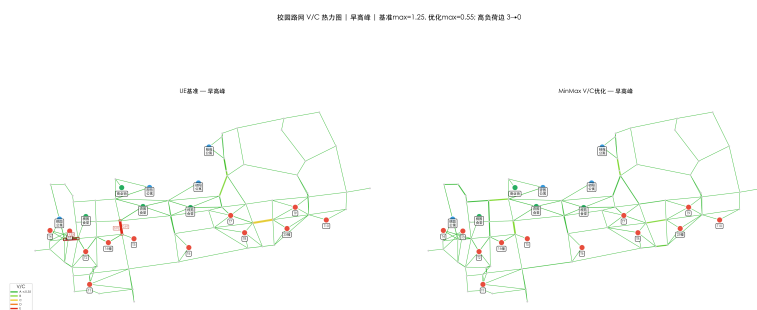


图 1-3: 早高峰路网 V/C 热力图 (左: UE 基准, 右: MinMax 增量分配优化)

图1-3显示, MinMax 优化后 N4→N14 路段 V/C 从 1.249 降至 0.255 (降幅 79.6%), 过饱和路段清零。

表 1-5: MinMax V/C 微观增量分配优化效果

时段	基准 max V/C	微观优化后 max V/C	降幅
早高峰	1.249	0.255	79.6%
午高峰	1.410	0.469	66.7%
晚高峰	1.251	0.700	44.1%

优化后全路网无 $V/C > 1.00$ 的过饱和路段。MinMax 输出包括每对 OD 在各方式下的具体分流路径。以早高峰 TYGY 至 T_16 的电动车出行为例，算法生成两条路径：

- **直达路径**（35 人）：TYGY → N4 → N14 → T_2 → N15 → T_16，全程较短但需穿过 N4→N14 窄路段（宽 6m）；
- **绕行路径**（228 人）：TYGY → N4 → T_4 → N14 → T_2 → N15 → T_16，经 T_4 绕行避开拥挤段。

86.7% 的电动车流量被分配至绕行路径。全路网 2284 条路径记录覆盖了所有关键 OD 对，可作为导流标识设置的数据基础。

方案对比与结论：综合对比方案 C（路网结构改造）与 MinMax 增量分流（路径分配优化）的效果：方案 C 虽能降低冲突风险 25.3%，但绕行诱发了新的拥堵点，过饱和路段从 1 条增至 2 条；而 MinMax 方案在不改动路网的条件下，将最大 V/C 从 1.249 降至 0.255，过饱和路段清零，且无任何路段恶化。因此，问题一的交通流优化最终采用 MinMax 方案作为核心优化手段，方案 C 的路网结构改造（单行线、人车隔离等）作为辅助治理建议，在问题四中进一步讨论。

二、 电动车承载力评估

2.1 问题重述

校园电动车数量持续增长，教学楼、宿舍区、食堂和商业街周边出现停车紧张、乱停放和通行干扰。问题二要求在不占用消防通道的前提下，结合道路宽度和建筑周边停车面积，测算校园可承载的最大电动车数量。结果将作为学校后续总量控制、分区停放和停车治理的依据。

2.2 问题分析

问题二采用“静态停车容量 + 动态道路容量 + 消防折减”的三层校核思路：先算建筑周边可停面积，再算道路接入能力，最后取两者较小值作为安全承载力。通过分区测算可以识别哪里是停车瓶颈、哪里是道路瓶颈，从而给出校园电动车总量控制上限。

2.3 模型假设

作如下假设：

1. 电动车停放限于建筑周边可规划区域，不占用消防通道、出入口、主通行人行道和绿化隔离带；
2. 每辆两轮电动车的基本占地面积取 $a_e = 1.08 \text{ m}^2$ ，并通过停车间距和通道系数 β 反映实际规范停放所需附加空间；
3. 各区域停车面积在短期内不发生结构性变化，安全折减系数可反映消防通道、疏散空间和不可停车空间的综合影响；
4. 动态道路容量由区域周边可接入道路的通行能力决定，并与问题一中得到的路网容量和高峰运行状态保持一致；
5. 校园电动车治理应同时区分高峰运行需求和车辆实际保有量，前者用于高峰运行校核，后者用于存量总量管理校核。

2.4 符号说明

表 2-1: 问题二主要符号说明

符号	说明	单位
A_i^{raw}	区域 i 原始候选停车面积	m^2
λ_i	区域 i 的消防安全折减系数	—
A_i^{eff}	区域 i 有效停车面积	m^2
a_e	单辆电动车基本占地面积	$\text{m}^2/\text{辆}$
β	规范停放附加空间系数	—
N_i^{static}	区域 i 静态停车容量	辆
$N_i^{dynamic}$	区域 i 动态道路容量	辆
N_i^*	区域 i 最终安全承载力	辆
N_{max}	全校电动车安全环境承载力	辆
D_i^{peak}	区域 i 高峰停车或运行需求	辆
ρ_i	区域 i 饱和度	—

2.5 模型的建立与求解

2.5.1 静态停车容量模型

根据《建筑设计防火规范》(GB 50016)对消防车道净宽度的强制性要求^[5],原始候选面积不能全部用于停车,需要严格扣除消防通道、建筑出入口、主通行人行道、绿化隔离带和必要疏散空间。为此,引入区域安全折减系数 λ_i ,得到有效停车面积:

$$A_i^{eff} = \lambda_i A_i^{raw}. \quad (2-1)$$

其中, $0 < \lambda_i < 1$ 。生活区和商业街人流密度较高、消防疏散要求更严格,折减系数相对较小;教学区周边若有较明确的开敞空间,折减系数可略高。

设单辆电动车基本占地面积为 a_e ,规范停放附加空间系数为 β ,则区域 i 的静态停车容量为

$$N_i^{static} = \left\lfloor \frac{A_i^{eff}}{\beta a_e} \right\rfloor. \quad (2-2)$$

考虑到常规两轮电动车的物理尺寸及《城市非机动车交通系统规划设计导则》的相关规定^[6],取其基本物理占地面积为 $a_e = 1.08 \text{ m}^2$;同时为保证车辆独立进出互不干扰,参考规范停放标准,附加空间系数取 $\beta = 1.35$,以计入必要的停车间隔、通行缝隙和实际管理中的无效冗余。

表 2-2: 消防安全折减后的静态容量测算结果

区域	原始面积/ m^2	折减系数	有效面积/ m^2	静态容量/辆
16 号楼教学区	630	0.72	450	308
25 号楼教学区	770	0.72	550	377
桃源生活区	13000	0.63	8160	5596
杏园生活区	7800	0.65	5100	3497
桂苑生活区	12000	0.63	7700	5281
东区新教学楼	3200	0.70	2200	1508
西区新教学楼	5700	0.70	4000	2743
商业街	1900	0.60	1200	823

2.5.2 动态道路容量模型

静态停车容量反映停放能力,动态道路容量反映周边道路接入能力。区域 i 的接入路段集合为 E_i ,路段 e 通行能力为 C_e ,电动车可分配比例为 η_b ,高峰持续时间 T_h ,动态道路容量为

$$N_i^{dynamic} = \sum_{e \in E_i} \eta_b C_e T_h. \quad (2-3)$$

该容量与问题一中路网参数和高峰运行状态衔接。

2.5.3 综合承载力模型

区域最终承载力由静态停车容量和动态道路容量共同约束,取二者较小值:

$$N_i^* = \min(N_i^{static}, N_i^{dynamic}). \quad (2-4)$$

全校电动车安全环境承载力为各区域承载力之和：

$$N_{max} = \sum_i N_i^*. \quad (2-5)$$

区域饱和度定义为

$$\rho_i = \frac{D_i^{peak}}{N_i^*}, \quad (2-6)$$

其中 D_i^{peak} 为区域高峰需求。 $\rho_i > 1$ 表示超载， $\rho_i \approx 1$ 表示饱和， $\rho_i < 1$ 表示容量充足。

表 2-3: 区域综合承载力与高峰需求校核结果

区域	N^{static}	$N^{dynamic}$	N^*	D_i	ρ
16 号楼教学区	308	1482	308	689	2.235
25 号楼教学区	377	1888	377	783	2.077
桃园生活区	5596	1888	1888	2334	1.237
杏园生活区	3497	1370	1370	1129	0.824
桂苑生活区	5281	709	709	1221	1.722
东区新教学楼	1508	1482	1482	311	0.210
西区新教学楼	2743	1793	1793	415	0.232
商业街	823	1482	823	2101	2.553

由表2-3，16 号楼、25 号楼、商业街和桂苑生活区四个区域超载（而非道路通行能力不足），桃园、杏园、桂苑生活区受动态道路容量约束。

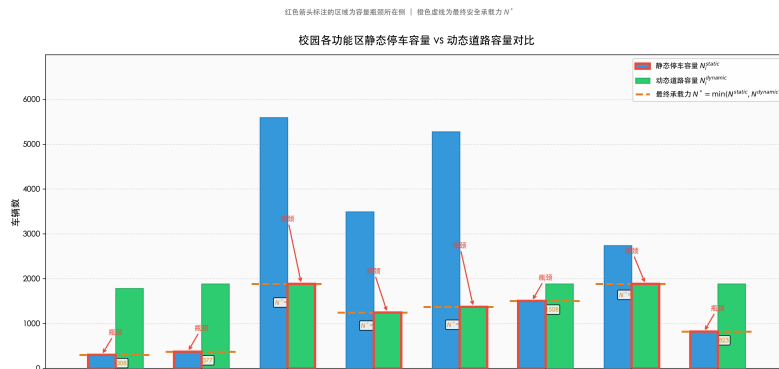


图 2-1: 各功能区静态停车容量与动态道路容量对比

2.5.4 全校承载力与双层饱和度校核

将各区域最终容量汇总，得到全校电动车安全环境承载力为

$$N_{max} = 9414 \text{ 辆}. \quad (2-7)$$

如果仅以模型估计的最差时段高峰运行需求进行校核，最大电动车运行需求为 5166 辆，则高峰运行饱和度为

$$\theta_{peak} = \frac{5166}{9414} = 0.549. \quad (2-8)$$

在 OD 模型口径下，高峰时段进入道路和停车系统的电动车规模未超过全校安全承载力。

高峰运行需求不等同于实际保有量。按公开资料中 11000 辆以上保有量校核：

$$\theta_{stock} = \frac{11000}{9414} \approx 1.168, \quad (2-9)$$

$$11000 - 9414 = 1586 \text{ 辆}. \quad (2-10)$$

按高峰 OD 需求校核，运行量未超过安全承载力；按实际保有量校核，存量已明显超过安全承载力。

2.5.5 敏感性检验：有效面积测量误差

承载力对区域有效停车面积 A_i^{eff} 的测量精度存在依赖。但需要注意，各区域的最终承载力 $N_i^* = \min(N_i^{static}, N_i^{dynamic})$ 由静态和动态容量中的较小者决定——在 8 个区域中，仅 16 号楼教学区、25 号楼教学区和商业街 3 个区域受静态面积约束，其余 5 个区域（桃园、杏园、桂苑、东区、西区）均受动态道路容量约束。因此， A_i^{eff} 的扰动并不会线性传导至 N_{max} 。

为检验结论稳健性，对 A_i^{eff} 施加 $\pm 20\%$ 均匀扰动，重新计算各区域 N_i^{static} 并与 $N_i^{dynamic}$ 取较小值，结果如表 2-4 所示。

表 2-4: N_{max} 对有效停车面积测量误差的敏感性

扰动幅度	N_{max} (辆)	ΔN_{max}	θ_{peak}
-20% (低估)	8809	-6.4%	0.586
-10%	9111	-3.2%	0.567
基准 (0%)	9414	-	0.549
+10%	9713	+3.2%	0.532
+20% (高估)	10015	+6.4%	0.516

结果表明，当 A_i^{eff} 扰动 $\pm 20\%$ 时，承载力变化仅约 $\pm 6.4\%$ ，且高峰运行饱和度 θ_{peak} 仍 < 1 。结论极具稳健性，不会改变“运行未超载、存量已超载”的定性结果。

2.5.6 问题二结论与治理建议

综合校核得出：（1）金华校区电动车安全承载力约 9414 辆；（2）高峰运行无整体拥堵（饱和度 0.549），但存量超载达 1.168；（3）空间失衡严重，16/25 号楼及商业街极度拥挤。治理建议实施“存量绝对控制 + 核心区禁停 + 外围缓冲限流”。

三、 微公交选址与调度

3.1 问题重述

为减少师生对私人电动车的依赖，学校计划投放校园微公交，缓解教学区、宿舍区、食堂和商业街之间的高峰出行压力。问题三要求基于校园空间拓扑确定接驳线路和站点，保证覆盖率的同时尽量不影响主干道通行。同时结合潮汐客流确定车辆投放量和发车时间表，在满足需求的前提下降低运营成本。

3.2 问题分析

问题三分别用最大覆盖选址模型确定站点，用路径规划模型确定行驶线路，用车辆调度模型确定投放量和发车间隔。其中站点选择强调覆盖需求和停靠安全，线路选择强调避开拥堵路段，调度部分则以最小化运营成本和等待成本为目标。

3.3 模型假设

作如下假设：

1. 接驳车主要服务校园内部短距离高峰出行，不考虑校外长距离客流；
2. 候选站点设置在宿舍区、核心教学楼、食堂或商业街周边，且停靠位置应满足道路宽度和安全条件；
3. 接驳车运行速度受道路拥堵影响，可用问题一中的 BPR 阻抗思想修正路段运行时间；
4. 微公交对私人电动车的替代需求由基础替代需求决定；
5. 接驳车按固定线路循环或往返运行，同一时段内车辆均匀发车，乘客平均等待时间约为发车间隔的一半；
6. 调度目标是在满足客流需求和最大发车间隔约束下，使运营成本、乘客等待成本和运能浪费成本之和最小。

3.4 符号说明

表 3-1: 问题三主要符号说明

符号	说明	单位
I	需求点集合	—
J	候选站点集合	—
x_j	候选站点 j 是否被选中	0-1
y_i	需求点 i 是否被覆盖	0-1
a_i	需求点 i 的需求权重	人次
r_{ij}	站点 j 是否覆盖需求点 i	0-1
P_j	候选站点 j 的适宜性得分	—
D_s^t	时段 t 的接驳需求	人次
D_{bike}^t	时段 t 的电动车运行需求	人次
N_{max}	校园电动车安全承载力	辆
n_t	时段 t 投放接驳车辆数	辆
K	单车载客量	人/车
τ_r	推荐线路单程运行时间	min
h_t	时段 t 发车间隔	min

3.5 模型的建立与求解

3.5.1 候选站点构造与适宜性评价

根据校园空间结构和问题二中识别出的高压区域，选取宿舍区、16 号楼、25 号楼、食堂和商业街周边作为候选站点。设候选站点集合为 J ，需求点集合为 I 。候选站点是否适宜停靠，需要同时考虑覆盖能力、道路宽度、拥堵程度和冲突风险。站点 j 对需求点 i 的覆盖关系定义为

$$r_{ij} = \begin{cases} 1, & d_{ij} \leq R_0, \\ 0, & d_{ij} > R_0, \end{cases} \quad (3-1)$$

其中 d_{ij} 为需求点 i 到候选站点 j 的步行距离， R_0 为最大可接受步行距离。

进一步构造候选站点适宜性得分：

$$P_j = \alpha_1 COV_j + \alpha_2 W_j - \alpha_3 VC_j - \alpha_4 RISK_j, \quad (3-2)$$

其中 COV_j 表示站点覆盖需求强度， W_j 表示周边道路宽度条件， VC_j 表示周边道路饱和度， $RISK_j$ 表示周边冲突风险。适宜性得分越高，说明该站点既能覆盖更多需求，又不容易因停靠造成额外拥堵或安全风险。

3.5.2 最大覆盖站点选址模型

在候选站点适宜性评价基础上，建立最大覆盖选址模型^[7]。设 x_j 表示候选站点 j 是否被选中， y_i 表示需求点 i 是否被覆盖， p 为可建设站点数量，则模型为

$$\begin{aligned} \max \quad & \sum_{i \in I} a_i y_i, \\ \text{s.t.} \quad & y_i \leq \sum_{j \in J} r_{ij} x_j, \quad \forall i \in I, \\ & \sum_{j \in J} x_j = p, \\ & x_j \in \{0, 1\}, \quad y_i \in \{0, 1\}. \end{aligned} \quad (3-3)$$

其中 a_i 为需求点权重，可由建筑容量、高峰 OD 需求和停车压力综合确定。模型目标是在给定站点数量下最大化被覆盖需求。计算结果表明，选取桃源站、16 号楼站、25 号楼站、杏园站和桂苑站能够较好覆盖宿舍区与核心教学区之间的潮汐客流，并兼顾食堂和生活区方向的出行需求。

3.5.3 基于 BPR 阻抗的线路规划模型

接驳车运行线路以 BPR 阻抗修正的通行时间为边权进行最短路搜索。沿用问题一中的 BPR 阻抗：

$$t_e^{bus} = \frac{l_e}{v_{bus}} \left[1 + \alpha \left(\frac{X_e}{C_e} \right)^\beta \right], \quad (3-4)$$

其中 v_{bus} 为接驳车自由流速度， X_e/C_e 为路段饱和度；阻抗参数同样沿用 $\alpha = 0.15$ ， $\beta = 4.0$ 以保持路网整体阻抗计算的一致性。

为综合比较不同线路，构造线路广义成本函数：

$$G_r = \omega_1 \frac{L_r}{L_{max}} + \omega_2 \frac{T_r}{T_{max}} + \omega_3 \overline{VC}_r + \omega_4 \overline{RISK}_r - \omega_5 \frac{Q_r}{Q_{max}}, \quad (3-5)$$

其中 L_r 为线路长度， T_r 为线路运行时间， $\overline{VC}_r$ 为线路平均饱和度， $\overline{RISK}_r$ 为线路平均冲突风险， Q_r 为线路覆盖需求。广义成本越小，线路综合表现越优。

候选线路评价结果如表3-2所示。R1 教学潮汐线长度和运行时间较短，广义成本最低，因此选为推荐线路。

表 3-2: 候选接驳线路评价结果

线路	类型	长度/m	时间/min	平均 V/C	覆盖需求	广义成本
R1	教学潮汐线	1243	5.8	0.149	51,300	0.4352
R2	校园环线	1661	7.9	0.180	51,300	0.6324

推荐线路为

$$\text{桃源站} \rightarrow \text{16 号楼站} \rightarrow \text{25 号楼站} \rightarrow \text{杏园站} \rightarrow \text{桂苑站}. \quad (3-6)$$

该线路能够连接主要宿舍区和核心教学区，适合承担早高峰宿舍到教学楼、午高峰教学楼到食堂及生活区、晚高峰教学楼返回宿舍的潮汐出行需求。

3.5.4 接驳需求识别模型

本文将时段 t 的接驳需求定义为

$$D_s^t = \eta_{base} D_{bike}^t, \quad (3-7)$$

其中 η_{base} 为基础绿色替代率。参考《2023 年中国主要城市通勤监测报告》中关于常规微循环公共交通体系对个体的分担率^[8]，结合校园接驳车的发车频次与舒适度预期，基准情景取 $\eta_{base} = 10\%$ 。由此得到早高峰、午高峰和晚高峰接驳需求分别为 447、517 和 454 人次。

3.5.5 潮汐客流调度模型

接驳车调度需要在满足客流需求的同时控制运营成本。设时段 t 持续时间为 H_t ，推荐线路单程运行时间为 τ_r ，投放车辆数为 n_t 。单车容量 K 取 25 人/车——参照现实校园接驳场景，中型客车在座位（约 15 座）基础上允许少量站立，实际容纳约 25 人，且车宽不足 2.5 m，可在校园最窄支路（宽 6 m）上正常通行。则发车间隔为

$$h_t = \frac{\tau_r}{n_t}, \quad (3-8)$$

时段总运能为

$$Q_t(n_t) = n_t K \frac{H_t}{\tau_r}. \quad (3-9)$$

综合考虑车辆运营成本、乘客等待成本和运能浪费成本，建立调度成本函数：

$$Z(n_t) = C_V n_t H_t + C_W D_s^t \frac{h_t}{2} + C_O \max(0, Q_t(n_t) - D_s^t), \quad (3-10)$$

约束条件为

$$Q_t(n_t) \geq D_s^t, \quad h_t \leq H_{max}, \quad n_t \in \mathbb{Z}^+. \quad (3-11)$$

其中第一项为车辆运营成本，第二项为乘客平均等待成本，第三项为运能闲置惩罚。通过枚举可行车辆数，选择总成本最小的 n_t 作为推荐投放量。

表 3-3: 基准情景下接驳车调度结果

时段	主方向	需求/人次	持续/min	单程/min	车辆数	间隔/min	满载率
早高峰	宿舍区 → 教学区	447	45	5.8	3	1.9	0.77
午高峰	教学区 → 食堂/商业街	517	50	5.8	3	1.9	0.80
晚高峰	教学区 → 宿舍/食堂	454	40	5.8	3	1.9	0.88

由表3-3可知，基准 10% 替代率下，三个时段各需投放 3 辆接驳车。各时段发车间隔均为 1.9 分钟，平均等待时间约 1.0 分钟，能够满足高峰期连续接驳需求。

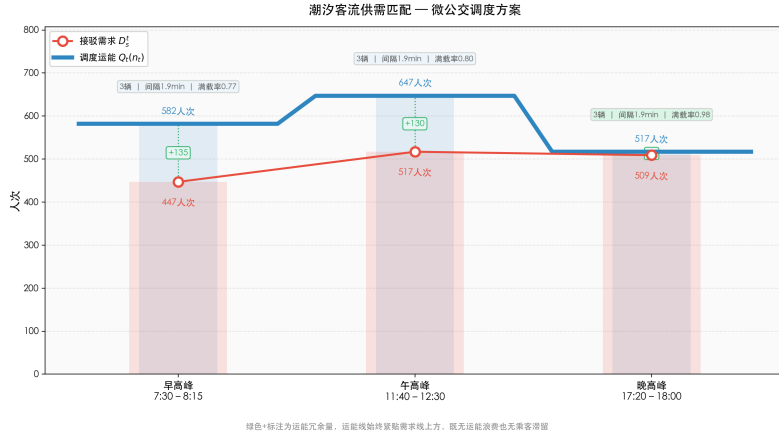


图 3-1: 潮汐客流供需匹配——微公交调度方案

3.5.6 敏感性检验：基础替代率扰动

为验证调度方案的鲁棒性，对核心参数基础绿色替代率 η_{base} 在 $[0.05, 0.25]$ 区间内以步长 0.05 进行敏感性扫描。此时接驳需求退化为

$$D_s^t(\eta_{base}) = \eta_{base} D_{bike}^t, \quad t \in \{\text{早, 午, 晚}\}, \quad (3-12)$$

代入调度模型重新求解各 η_{base} 下的最优投放方案与成本构成，结果如表3-4所示。

表 3-4: 接驳车调度对基础替代率 η_{base} 的敏感性

η_{base}	早高峰 n_t	午高峰 n_t	晚高峰 n_t	总成本	满载率	冗余率
5%	2	2	2	2886	0.64	36.2%
10%	3	3	3	4149	0.85	15.0%
15%	4	4	5	5593	0.88	11.7%
20%	5	5	6	6713	0.95	4.5%
25%	6	6	8	8126	0.96	4.0%

当 η_{base} 扰动至 25% 时，运能平稳自适应（仅需调配备用车），满载率提升至 0.96。说明模型能平滑应对需求激增，基于复合成本的调度机制非常稳健。

四、 综合治理建议

4.1 问题重述

问题四要求把前三问的模型结果转化成一页纸建议书，用通俗语言向管理层说明治理方案。建议书需要回答校园最多能容纳多少非机动车，以及为什么要同步推进路网优化、停车治理和微公交接驳。本问的重点不是重新建模，而是把前面得到的结论整理成可执行、可落地的管理建议。

4.2 问题分析

问题四主要是结果整合与表达转换：把问题一的路网优化结果、问题二的承载力上限、问题三的接驳方案统一整理成管理建议。写作上采用“安全优先 + 总量控制 + 微公交替代”的思路，把定量结论转成学校可以直接执行的措施清单。

4.3 模型假设

作如下假设：

1. 学校管理目标以安全优先为基本原则，在安全风险明显下降的前提下，可以接受小幅通行时间增加；
2. 校园电动车治理既要考虑高峰同时运行需求，也要考虑实际保有量形成的长期存量压力；
3. 微公交主要用于替代高峰私人电动车直达需求，不能单独吸收全部存量电动车；

4.4 符号说明

表 4-1: 问题四主要符号说明

符号	说明	单位
N_{max}	校园电动车安全环境承载力	辆
N_{stock}	校园电动车实际保有量估计值	辆
θ_{peak}	高峰运行饱和度	—
θ_{stock}	存量饱和度	—
TTT	全路网总行程时间	人·s
S	人车冲突风险指标	—
D_s^t	时段 t 微公交接驳需求	人次
$Priority_k$	治理措施 k 的综合优先级得分	—

4.5 模型的建立与求解

4.5.1 接驳效果评价与问题三结论

将接驳车替代的电动车流量从原高峰 OD 中扣除，并重新进行交通分配，可得到微公交实施后的路网改善效果，如表4-2所示。

表 4-2: 微公交接驳后的交通改善效果

时段	电动车削减率	TTT 优化前	TTT 优化后	TTT 改善	冲突风险改善
早高峰	10.8%	2,557,530	2,488,595	2.7%	3.8%

基准 10% 替代率下，微公交使早高峰 TTT 下降 2.7%， S 下降 3.8%，电动车出行削减 10.8%。接驳车本身占用道路资源，改善幅度受替代率限制。微公交的核心作用是减少私人电动车直达教学区、食堂和商业街的流量，分流核心区交通压力。

问题三推荐教学潮汐接驳线：桃园站—16 号楼站—25 号楼站—杏园站—桂苑站。早高峰和午高峰各 3 辆，晚高峰 3 辆。微公交与问题一的 MinMax 分流优化及问题二的核心区限停、总量控制共同构成综合治理方案。

4.5.2 前三问核心结果汇总

根据前述模型测算：第一，MinMax 增量分流算法在不改动道路的情况下，通过动态阻抗将瓶颈路段饱和度降低 79.6%，并输出了每对 OD 的具体分流路径，证明拥堵主因在于路径集中而非路网容量不足。第二，全校电动车安全承载力 $N_{max} = 9414$ 的上限揭示出当前保有量（约 11000 辆）的存量饱和度达到 1.168，校园微交通长期面临超过 1586 辆的停放空间缺口，这是拥挤失序的本源。第三，以“桃园站—16 号楼站—25 号楼站—杏园站—桂苑站”为轴的潮汐接驳线只需投入三拨 3 辆校园公交，即可在高峰削减 10.8% 的电动车需求，为校园路网减负。

4.5.3 治理措施优先级评价框架

设治理措施 k 的综合优先级得分为

$$Priority_k = 0.40Safety_k + 0.35Capacity_k + 0.25Feasibility_k, \quad (4-1)$$

其中 $Safety_k$ 表示安全改善程度， $Capacity_k$ 表示容量缓解程度， $Feasibility_k$ 表示实施可行性。权重设置体现安全优先、兼顾容量和可操作性。

表 4-3: 校园微交通治理措施优先级排序

治理措施	安全改善	容量缓解	可行性	优先级
清理消防通道、划定停车边界	高	高	高	第一优先
16 号楼、25 号楼、商业街核心区限停	高	高	中	第一优先
开通教学潮汐微公交线路	中	中	高	第二优先
主干道人车隔离与分时限行	高	中	中	第二优先
注册准入与存量车辆分期管控	高	高	中	长期推进
外围集中停放与步行接驳引导	中	高	中	长期推进

由表4-3，第一优先为消防通道清理、停车边界划定和核心区限停；第二优先为教学潮汐微公交及主干道人车隔离、分时限行；长期推进注册准入与存量管控。

4.5.4 面向管理层的一页纸建议书

关于浙师大金华校区校园“微交通”综合治理的建议书

基于对校园道路、人车混行、电动车停放和微公交接驳系统的综合建模分析，我们建议学校采取”安全优先、总量控制、核心区限停、外围分流、公交替代”的综合治理策略，具体建议如下：

明确承载上限，逐步控制电动车总量。经测算，在严格保障消防与疏散通道的前提下，金华校区电动车安全承载力约为 9414 辆。当前校园实际保有量（超 11000 辆）已产生约 1586 辆的停放空间缺口。校园拥堵与乱停放的根源在于车辆存量已明显超出管理能力，建议学校以此承载力为参考，建立实名注册准入制度，逐步消化超载存量。

实施路网动态分流引导，降低冲突风险。模型评估显示，MinMax V/C 增量分流算法在不改动路网结构的条件下，将瓶颈路段饱和度从 1.249 降至 0.255（降幅 79.6%），全网过饱和路段清零。建议基于模型输出的 2284 条最优分流路径，在关键路口设置导流标识，引导电动车流避开 N4→N14 等拥堵瓶颈，实现车流的”自适应铺摊”。

设立核心禁停区，引导外围停车。16 号楼、25 号楼及商业街周边是目前校园停车矛盾最突出的区域。建议将这些区域设为核心限停区，严禁占用建筑出入口和主通道。同时，利用东区、西区新教学楼周边相对充足的容量作为缓冲，引导师生养成”外围停车、步行接驳进入核心区”的习惯。

投放潮汐微公交，分担高峰直达车流。建议开通连接主要生活区与核心教学区的微公交专线（如桃源站—16/25 号楼—杏园—桂苑）。在早、中、晚高峰各投入 3 辆接驳车，即可满足高频连续发车需求。此举虽不能替代全部存量电动车，但能切实减少高峰期私人电动车直达核心区的规模，进一步缓解路网压力。

分阶段稳步推进治理。建议短期内以”清理消防通道、实施核心区限停”为抓手；中期配套微公交线路及候车设施；长期则落实电动车注册准入与分区管理。通过软硬结合，可在不大规模改造校园道路的前提下，切实提升校园交通的安全与有序。

4.5.5 问题四结论

综合前三问结果：路网优化的重点是降低冲突风险，电动车存量已超过安全承载力，微公交承担部分高峰替代和核心区分流功能。**管理建议：**以 9414 辆为承载力参考上限，核心区限停和消防通道清理为近期重点，教学潮汐微公交为中期工具，注册准入和存量管控为长期手段。

五、 模型评价与改进

5.1 模型优点

1. 模型覆盖路网优化（UE）、承载力评估（静态/动态）和微公交规划（MCLP 选址 + 调度），各问题间数据（路网、OD、容量）相互衔接。
2. 引入人车冲突风险指标 S ，将人车混行安全隐患量化为可优化的指标。
3. MinMax 增量分流模型能输出每对 OD 的具体绕行路径和流量分配，为导流标识设置和出行推荐提供可直接落地的数据支撑。

5.2 模型缺点

1. 引力模型替代实测 OD 数据，距离阶梯方式划分未考虑坡度、遮阳等因素。
2. BPR 参数 ($\alpha = 0.15, \beta = 4.0$) 为标准城市快速路取值，校园低速混行环境需实测标定。
3. 静态 UE 分配假设时段内需求均匀分布，低估了时段初的峰值拥堵效应。
4. 有效停车面积 A_i^{eff} 依赖暂估值， λ_i 和原始候选面积需实地测量验证。
5. MCLP 选址为静态模型，未考虑站点布局随学期规划或临时活动变化的动态调整需求。

5.3 模型推广

本文模型可直接推广至具有类似教学-生活分区结构、以低速混行为主的其他大学校园或大型封闭园区，仅需替换路网数据和建筑容量参数。若引入实时导航 App，UE 假设需升级为随机用户均衡（SUE）或动态用户均衡（DUE）；若考虑电动车充电约束，OD 模型需引入充电站节点；若治理涉及价格杠杆，决策变量需从 0-1 扩展为连续变量。

参考文献

- [1] 王炜, 陈学武. 交通规划 [M]. 3 版. 北京: 人民交通出版社, 2017.
- [2] Bureau of Public Roads. Traffic Assignment Manual[R]. Washington, DC: U.S. Department of Commerce, 1964.
- [3] 王炜, 曲大义, 朱中. 城市交通网络综合平衡交通分配模型研究 [J]. 东南大学学报 (自然科学版), 2000, (01): 117-120.
- [4] Frank M, Wolfe P. An algorithm for quadratic programming[J]. Naval Research Logistics Quarterly, 1956, 3(1-2): 95-110.
- [5] 中华人民共和国住房和城乡建设部, 中华人民共和国国家质量监督检验检疫总局. GB 50016-2014 建筑设计防火规范 (2018 年版)[S]. 北京: 中国计划出版社, 2018.
- [6] 住房和城乡建设部. 城市非机动车交通系统规划设计导则 [S]. 北京: 中国建筑工业出版社, 2021.
- [7] Church R, ReVelle C. The maximal covering location problem[J]. Papers of the Regional Science Association, 1974, 32(1): 101-118.

说明：本文参考文献的检索与整理工作由深度求索公司（DeepSeek v4 pro）辅助完成。

附录

附录目录

1. 附录 A: 校园路网原始数据 (campus_network_with_coords.txt)
2. 附录 B: 步骤一: 校园路网构建 (step1_build_graph.py)
3. 附录 C: 步骤二: OD 需求矩阵生成 (step2_od_matrix.py)
4. 附录 D: 步骤三: 多方式用户均衡分配 (step3_ue_fw.py)
5. 附录 E: 步骤四: 交通组织方案设计与对比 (step4_scenarios.py)
6. 附录 F: 步骤五: MinMax V/C 增量分流 (step5_k_shortest.py)
7. 附录 G: 步骤六: 方案评价与报告输出 (step6_evaluate.py)
8. 附录 H: 步骤七: V/C 热力图生成 (step7_heatmap.py)
9. 附录 I: 步骤八: 功能区域划分 (step8_zones.py)
10. 附录 J: 步骤九: 静态停车容量计算 (step9_static.py)
11. 附录 K: 步骤十: 动态道路容量计算 (step10_dynamic.py)
12. 附录 L: 步骤十一: 综合承载力求解 (step11_solve.py)
13. 附录 M: 步骤十二: 问题三数据准备 (step12_prepare.py)
14. 附录 N: 步骤十三: 候选站点构造与评价 (step13_candidate_stops.py)
15. 附录 O: 步骤十四: MCLP 最大覆盖选址 (step14_coverage.py)
16. 附录 P: 步骤十五: BPR 阻抗线路规划 (step15_routes.py)
17. 附录 Q: 步骤十六: 接驳需求识别与调度 (step16_bus_demand.py)
18. 附录 R: 步骤十七: 微公交效果评价 (step17_effect.py)

附录 A: 校园路网原始数据 (campus_network_with_coords.txt)

N1, 交叉口
N2, 交叉口
N3, 交叉口
N4, 交叉口
N5, 交叉口
N6, 交叉口
N7, 交叉口
N8, 交叉口
N9, 交叉口
N10, 交叉口
N11, 交叉口
N12, 交叉口
N13, 交叉口
N14, 交叉口
N15, 交叉口
N16, 交叉口
N17, 交叉口
N18, 交叉口
N19, 交叉口
N20, 交叉口
N21, 交叉口
N22, 交叉口
N23, 交叉口
N24, 交叉口
N25, 交叉口
N26, 交叉口
N27, 交叉口

N28, 交叉口
 N29, 交叉口
 N30, 交叉口
 N31, 交叉口
 N32, 交叉口
 N33, 交叉口
 N34, 交叉口
 N35, 交叉口
 N36, 交叉口
 N37, 交叉口
 N38, 交叉口
 N39, 交叉口
 N40, 交叉口
 N41, 交叉口
 N42, 交叉口
 N43, 交叉口
 N44, 交叉口
 N45, 交叉口
 N46, 交叉口
 N47, 交叉口
 N48, 交叉口
 N49, 交叉口
 N50, 交叉口
 N51, 交叉口
 N52, 交叉口
 N53, 交叉口
 N54, 交叉口
 N55, 交叉口
 N56, 交叉口
 N57, 交叉口
 N58, 交叉口
 N59, 交叉口
 N60, 交叉口
 N61, 交叉口
 N62, 交叉口
 N63, 交叉口
 N64, 交叉口
 N1, N2, 81.7, 12.0, 119.62817277, 29.13458115, 119.62900939, 29.13465612
 N2, N3, 53.2, 12.0, 119.62900939, 29.13465612, 119.62899866, 29.13513406
 N2, N5, 114.9, 12.0, 119.62900939, 29.13465612, 119.62984602, 29.13392515
 N4, N3, 82.8, 6.0, 119.62984602, 29.13521840, 119.62899866, 29.13513406
 N4, N5, 143.8, 12.0, 119.62984602, 29.13521840, 119.62984602, 29.13392515
 N4, N14, 73.1, 6.0, 119.62984602, 29.13521840, 119.63059683, 29.13526526
 N6, N5, 52.1, 6.0, 119.63027505, 29.13364400, 119.62984602, 29.13392515
 N6, N14, 183.0, 12.0, 119.63027505, 29.13364400, 119.63059683, 29.13526526
 N6, N26, 113.3, 6.0, 119.63027505, 29.13364400, 119.63144093, 29.13362358
 N7, N4, 354.4, 12.0, 119.62905229, 29.13832964, 119.62984602, 29.13521840
 N8, N7, 78.1, 8.0, 119.62985674, 29.13833901, 119.62905229, 29.13832964
 N9, N8, 83.6, 6.0, 119.63029651, 29.13769241, 119.62985674, 29.13833901
 N9, N10, 39.8, 12.0, 119.63029651, 29.13769241, 119.63033941, 29.13733630
 N10, N11, 121.2, 6.0, 119.63033941, 29.13733630, 119.63158362, 29.13742064
 N10, N14, 231.6, 12.0, 119.63033941, 29.13733630, 119.63059683, 29.13526526
 N11, N12, 109.8, 8.0, 119.63158362, 29.13742064, 119.63270985, 29.13750498
 N11, N15, 229.4, 8.0, 119.63158362, 29.13742064, 119.63165870, 29.13535897
 N12, N28, 291.6, 8.0, 119.63270985, 29.13750498, 119.63571071, 29.13741171
 N13, N12, 126.1, 6.0, 119.63272058, 29.13863889, 119.63270985, 29.13750498
 N14, N15, 103.7, 12.0, 119.63059683, 29.13526526, 119.63165870, 29.13535897
 N15, N16, 140.6, 12.0, 119.63165870, 29.13535897, 119.63309598, 29.13550891
 N15, N27, 184.4, 8.0, 119.63165870, 29.13535897, 119.63168227, 29.13370089
 N16, N18, 221.4, 12.0, 119.63309598, 29.13550891, 119.63347769, 29.13354604
 N16, N30, 85.2, 8.0, 119.63309598, 29.13550891, 119.63298055, 29.13626843
 N17, N6, 55.0, 12.0, 119.63034034, 29.13315244, 119.63027505, 29.13364400
 N17, N23, 97.9, 6.0, 119.63034034, 29.13315244, 119.63040288, 29.13227407
 N18, N19, 207.8, 12.0, 119.63347769, 29.13354604, 119.63377645, 29.13169537
 N18, N25, 196.2, 8.0, 119.63347769, 29.13354604, 119.63147579, 29.13331197
 N18, N45, 278.2, 8.0, 119.63347769, 29.13354604, 119.63630554, 29.13394389
 N19, N20, 77.7, 12.0, 119.63377645, 29.13169537, 119.63370135, 29.13239122
 N20, N18, 130.2, 6.0, 119.63370135, 29.13239122, 119.63347769, 29.13354604

N20, N21, 186.6, 12.0, 119.63370135, 29.13239122, 119.63180004, 29.13214990
 N21, N22, 46.6, 12.0, 119.63180004, 29.13214990, 119.63136505, 29.13232774
 N22, N24, 100.9, 12.0, 119.63136505, 29.13232774, 119.63125322, 29.13322997
 N23, N22, 93.6, 6.0, 119.63040288, 29.13227407, 119.63136505, 29.13232774
 N24, N17, 89.1, 12.0, 119.63125322, 29.13322997, 119.63034034, 29.13315244
 N24, N25, 23.5, 6.0, 119.63125322, 29.13322997, 119.63147579, 29.13331197
 N24, N27, 66.9, 6.0, 119.63125322, 29.13322997, 119.63168227, 29.13370089
 N26, N25, 34.8, 8.0, 119.63144093, 29.13362358, 119.63147579, 29.13331197
 N26, N27, 25.0, 8.0, 119.63144093, 29.13362358, 119.63168227, 29.13370089
 N28, N29, 95.6, 8.0, 119.63571071, 29.13741171, 119.63583942, 29.13655893
 N28, N31, 283.2, 12.0, 119.63571071, 29.13741171, 119.63861133, 29.13767367
 N28, N44, 210.5, 6.0, 119.63571071, 29.13741171, 119.63608029, 29.13554640
 N29, N30, 279.5, 6.0, 119.63583942, 29.13655893, 119.63298055, 29.13626843
 N30, N12, 140.0, 12.0, 119.63298055, 29.13626843, 119.63270985, 29.13750498
 N31, N32, 71.1, 6.0, 119.63861133, 29.13767367, 119.63875077, 29.13704580
 N31, N33, 135.1, 12.0, 119.63861133, 29.13767367, 119.63905109, 29.13882631
 N33, N34, 115.9, 12.0, 119.63905109, 29.13882631, 119.63999498, 29.13946353
 N33, N46, 206.8, 12.0, 119.63905109, 29.13882631, 119.64011325, 29.13721448
 N33, N61, 109.2, 8.0, 119.63905109, 29.13882631, 119.63887631, 29.13979664
 N34, N35, 300.3, 12.0, 119.63999498, 29.13946353, 119.64300926, 29.13886379
 N34, N58, 232.1, 6.0, 119.63999498, 29.13946353, 119.63962075, 29.14152513
 N35, N37, 151.5, 12.0, 119.64300926, 29.13886379, 119.64366355, 29.13762681
 N35, N57, 98.8, 6.0, 119.64300926, 29.13886379, 119.64348210, 29.13965095
 N36, N35, 126.3, 6.0, 119.64359919, 29.13785172, 119.64300926, 29.13886379
 N36, N37, 25.8, 6.0, 119.64359919, 29.13785172, 119.64366355, 29.13762681
 N36, N38, 137.7, 6.0, 119.64359919, 29.13785172, 119.64367427, 29.13661472
 N37, N38, 112.5, 8.0, 119.64366355, 29.13762681, 119.64367427, 29.13661472
 N37, N47, 202.2, 12.0, 119.64366355, 29.13762681, 119.64159344, 29.13743002
 N37, N52, 210.5, 8.0, 119.64366355, 29.13762681, 119.64580964, 29.13788920
 N38, N48, 200.4, 6.0, 119.64367427, 29.13661472, 119.64163634, 29.13633359
 N39, N38, 270.4, 8.0, 119.64171142, 29.13489040, 119.64367427, 29.13661472
 N39, N40, 65.1, 6.0, 119.64171142, 29.13489040, 119.64104641, 29.13481543
 N39, N41, 146.0, 8.0, 119.64171142, 29.13489040, 119.64022051, 29.13472172
 N39, N48, 160.6, 6.0, 119.64171142, 29.13489040, 119.64163634, 29.13633359
 N41, N40, 80.9, 6.0, 119.64022051, 29.13472172, 119.64104641, 29.13481543
 N41, N42, 226.0, 8.0, 119.64022051, 29.13472172, 119.63831129, 29.13588376
 N41, N45, 390.0, 6.0, 119.64022051, 29.13472172, 119.63630554, 29.13394389
 N42, N43, 112.5, 8.0, 119.63831129, 29.13588376, 119.63800024, 29.13685837
 N42, N44, 219.9, 8.0, 119.63831129, 29.13588376, 119.63608029, 29.13554640
 N42, N49, 214.5, 6.0, 119.63831129, 29.13588376, 119.64049939, 29.13614616
 N43, N29, 212.5, 8.0, 119.63800024, 29.13685837, 119.63583942, 29.13655893
 N43, N32, 75.8, 8.0, 119.63800024, 29.13685837, 119.63875077, 29.13704580
 N44, N29, 115.0, 6.0, 119.63608029, 29.13554640, 119.63583942, 29.13655893
 N45, N44, 179.5, 8.0, 119.63630554, 29.13394389, 119.63608029, 29.13554640
 N46, N32, 133.7, 8.0, 119.64011325, 29.13721448, 119.63875077, 29.13704580
 N46, N47, 240, 12.0, 119.64011325, 29.13721448, 119.64159344, 29.13743002
 N46, N49, 124.6, 12.0, 119.64011325, 29.13721448, 119.64049939, 29.13614616
 N47, N48, 122.0, 12.0, 119.64159344, 29.13743002, 119.64163634, 29.13633359
 N49, N40, 157.2, 6.0, 119.64049939, 29.13614616, 119.64104641, 29.13481543
 N49, N48, 112.4, 12.0, 119.64049939, 29.13614616, 119.64163634, 29.13633359
 N50, N38, 207.1, 6.0, 119.64577746, 29.13692397, 119.64367427, 29.13661472
 N52, N50, 107.4, 6.0, 119.64580964, 29.13788920, 119.64577746, 29.13692397
 N52, N51, 130.6, 6.0, 119.64580964, 29.13788920, 119.64712893, 29.13811411
 N52, N53, 296.5, 8.0, 119.64580964, 29.13788920, 119.64520898, 29.14050371
 N54, N53, 232.4, 8.0, 119.64426510, 29.14242472, 119.64520898, 29.14050371
 N54, N55, 198.8, 8.0, 119.64426510, 29.14242472, 119.64225934, 29.14206863
 N55, N58, 263.3, 8.0, 119.64225934, 29.14206863, 119.63962075, 29.14152513
 N56, N55, 74.0, 6.0, 119.64213063, 29.14141268, 119.64225934, 29.14206863
 N56, N57, 235.8, 6.0, 119.64213063, 29.14141268, 119.64348210, 29.13965095
 N57, N53, 192.7, 6.0, 119.64348210, 29.13965095, 119.64520898, 29.14050371
 N58, N59, 88.8, 8.0, 119.63962075, 29.14152513, 119.63877340, 29.14122526
 N59, N61, 159.2, 8.0, 119.63877340, 29.14122526, 119.63887631, 29.13979664
 N60, N33, 32.7, 8.0, 119.63898885, 29.13911512, 119.63905109, 29.13882631
 N60, N61, 76.6, 8.0, 119.63898885, 29.13911512, 119.63887631, 29.13979664
 N62, N59, 22.5, 6.0, 119.63874760, 29.14102423, 119.63877340, 29.14122526
 N62, N61, 137.1, 6.0, 119.63874760, 29.14102423, 119.63887631, 29.13979664
 N30, N63, 140, 6.0, 119.63298055, 29.13626843, 119.63160753, 29.13622825
 N63, N64, 110, 6.0, 119.63160753, 29.13622825, 119.63052392, 29.13619077

SYJ, N13, 50, 99, 119.63307610, 29.13815514, 119.63272058, 29.13863889
 TYGY, N3, 50, 99, 119.62954747, 29.13636994, 119.62899866, 29.13513406
 TYGY, N4, 50, 99, 119.62954747, 29.13636994, 119.62984602, 29.13521840
 TYGY, N7, 50, 99, 119.62954747, 29.13636994, 119.62905229, 29.13832964
 TYST, N14, 30, 99, 119.63104886, 29.13651519, 119.63059683, 29.13526526
 TYST, N14, 30, 99, 119.63104886, 29.13651519, 119.63059683, 29.13526526
 XYST, N30, 50, 99, 119.63428269, 29.13708215, 119.63298055, 29.13626843
 XYST, N12, 50, 99, 119.63428269, 29.13708215, 119.63270985, 29.13750498
 XYST, N28, 50, 99, 119.63428269, 29.13708215, 119.63571071, 29.13741171
 XYST, N29, 50, 99, 119.63428269, 29.13708215, 119.63583942, 29.13655893
 XYGY, N13, 50, 99, 119.63465289, 29.13815983, 119.63272058, 29.13863889
 XYGY, N12, 50, 99, 119.63465289, 29.13815983, 119.63270985, 29.13750498
 GYST, N28, 50, 99, 119.63696702, 29.13702391, 119.63571071, 29.13741171
 GYST, N29, 50, 99, 119.63696702, 29.13702391, 119.63583942, 29.13655893
 GYST, N31, 50, 99, 119.63696702, 29.13702391, 119.63861133, 29.13767367
 GYST, N32, 50, 99, 119.63696702, 29.13702391, 119.63875077, 29.13704580
 CYGY, N28, 50, 99, 119.63741735, 29.13842020, 119.63571071, 29.13741171
 GYGY, N61, 50, 99, 119.63801779, 29.14045370, 119.63887631, 29.13979664
 GYGY, N62, 50, 99, 119.63801779, 29.14045370, 119.63874760, 29.14102423
 T_16, N15, 30, 99, 119.63231395, 29.13503921, 119.63165870, 29.13535897
 T_16, N16, 30, 99, 119.63231395, 29.13503921, 119.63309598, 29.13550891
 T_25, N48, 50, 99, 119.64243197, 29.13583678, 119.64163634, 29.13633359
 T_25, N39, 50, 99, 119.64243197, 29.13583678, 119.64171142, 29.13489040
 T_25, N38, 50, 99, 119.64243197, 29.13583678, 119.64367427, 29.13661472
 T_1, N25, 50, 99, 119.63125996, 29.13267291, 119.63147579, 29.13331197
 T_1, N24, 50, 99, 119.63125996, 29.13267291, 119.63125322, 29.13322997
 T_1, N22, 50, 99, 119.63125996, 29.13267291, 119.63136505, 29.13232774
 T_1, N18, 50, 99, 119.63125996, 29.13267291, 119.63347769, 29.13354604
 T_1, N20, 50, 99, 119.63125996, 29.13267291, 119.63370135, 29.13239122
 T_2, N14, 50, 99, 119.63103961, 29.13452948, 119.63059683, 29.13526526
 T_2, N15, 50, 99, 119.63103961, 29.13452948, 119.63165870, 29.13535897
 T_2, N6, 50, 99, 119.63103961, 29.13452948, 119.63027505, 29.13364400
 T_2, N27, 50, 99, 119.63103961, 29.13452948, 119.63168227, 29.13370089
 T_3, N3, 50, 99, 119.63012757, 29.13569152, 119.62899866, 29.13513406
 T_3, N4, 50, 99, 119.63012757, 29.13569152, 119.62984602, 29.13521840
 T_3, N5, 50, 99, 119.63012757, 29.13569152, 119.62984602, 29.13392515
 T_3, N2, 50, 99, 119.63012757, 29.13569152, 119.62900939, 29.13465612
 T_4, N4, 50, 99, 119.62901829, 29.13572332, 119.62984602, 29.13521840
 T_4, N14, 50, 99, 119.62901829, 29.13572332, 119.63059683, 29.13526526
 T_5, N16, 50, 99, 119.63377062, 29.13525944, 119.63309598, 29.13550891
 T_6, N44, 50, 99, 119.63687288, 29.13475673, 119.63608029, 29.13554640
 T_6, N45, 50, 99, 119.63687288, 29.13475673, 119.63630554, 29.13394389
 T_7, N42, 50, 99, 119.63924121, 29.13657711, 119.63831129, 29.13588376
 T_7, N43, 50, 99, 119.63924121, 29.13657711, 119.63800024, 29.13685837
 T_7, N32, 50, 99, 119.63924121, 29.13657711, 119.63875077, 29.13704580
 T_7, N46, 50, 99, 119.63924121, 29.13657711, 119.64011325, 29.13721448
 T_7, N49, 50, 99, 119.63924121, 29.13657711, 119.64049939, 29.13614616
 T_8, N42, 50, 99, 119.64003742, 29.13560250, 119.63831129, 29.13588376
 T_8, N41, 50, 99, 119.64003742, 29.13560250, 119.64022051, 29.13472172
 T_8, N49, 50, 99, 119.64003742, 29.13560250, 119.64049939, 29.13614616
 T_8, N40, 50, 99, 119.64003742, 29.13560250, 119.64104641, 29.13481543
 T_9, N47, 50, 99, 119.64290324, 29.13704265, 119.64159344, 29.13743002
 T_9, N37, 50, 99, 119.64290324, 29.13704265, 119.64366355, 29.13762681
 T_9, N38, 50, 99, 119.64290324, 29.13704265, 119.64367427, 29.13661472
 T_9, N48, 50, 99, 119.64290324, 29.13704265, 119.64163634, 29.13633359
 T_10, N38, 50, 99, 119.64466033, 29.13632409, 119.64367427, 29.13661472
 T_10, N50, 50, 99, 119.64466033, 29.13632409, 119.64577746, 29.13692397

附录 B：步骤一：校园路网构建 (step1_build_graph.py)

```
1 import networkx as nx
2 import pandas as pd
3 import numpy as np
4 import os, re
5 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```



```

6 INPUT_FILE = os.path.join(BASE_DIR, 'campus_network_with_coords.txt')
7 OUTPUT_DIR = os.path.join(BASE_DIR, 'output_step1')
8 os.makedirs(OUTPUT_DIR, exist_ok=True)
9 BUILDING_TYPE = {
10     'T_16': '教学楼', 'T_25': '教学楼',
11     'T_1': '教学楼', 'T_2': '教学楼', 'T_3': '教学楼',
12     'T_4': '教学楼', 'T_5': '教学楼', 'T_6': '教学楼',
13     'T_7': '教学楼', 'T_8': '教学楼', 'T_9': '教学楼', 'T_10': '教学楼',
14     'TYGY': '宿舍', 'XYGY': '宿舍', 'CYGY': '宿舍', 'GYGY': '宿舍',
15     'TYST': '食堂', 'XYST': '食堂', 'GYST': '食堂',
16     'SYJ': '商业街',
17 }
18 CAPACITY = {
19     'T_16': 3000, 'T_25': 2000,
20     'T_1': 1000, 'T_2': 1000, 'T_3': 1000, 'T_4': 1000, 'T_5': 1000,
21     'T_6': 1000, 'T_7': 1000, 'T_8': 1000, 'T_9': 1000, 'T_10': 1000,
22     'TYGY': 13000, 'XYGY': 7000, 'CYGY': 3000, 'GYGY': 5000,
23     'TYST': 1100, 'XYST': 1100, 'GYST': 1100,
24     'SYJ': 5000,
25 }
26 def classify_road(width):
27     if width >= 99:
28         return 9999, 5.56
29     elif width >= 12:
30         return 1200, 8.33
31     elif width >= 8:
32         return 800, 6.94
33     elif width >= 6:
34         return 600, 5.56
35     else:
36         return 300, 4.17
37 def parse_network(filepath):
38     nodes = {}
39     edges = []
40     current_section = None
41     with open(filepath, 'r', encoding='utf-8') as f:
42         for line in f:
43             line = line.strip()
44             if not line or line.startswith('#'):
45                 if '节点清单' in line:
46                     current_section = 'nodes'
47                 elif '边清单' in line:
48                     current_section = 'edges'
49                 continue
50             if current_section == 'nodes':
51                 parts = line.split(',')
52                 name = parts[0].strip()
53                 ntype = parts[1].strip()
54                 nodes[name] = {'type': ntype}
55                 if len(parts) >= 4:

```

```

56         try:
57             nodes[name]['lon'] = float(parts[2].strip())
58             nodes[name]['lat'] = float(parts[3].strip())
59         except:
60             pass
61     elif current_section == 'edges':
62         parts = line.split(',')
63         u = parts[0].strip()
64         v = parts[1].strip()
65         length = float(parts[2].strip())
66         width = float(parts[3].strip())
67         lon1 = float(parts[4].strip()) if len(parts) >= 5 else 0
68         lat1 = float(parts[5].strip()) if len(parts) >= 6 else 0
69         lon2 = float(parts[6].strip()) if len(parts) >= 7 else 0
70         lat2 = float(parts[7].strip()) if len(parts) >= 8 else 0
71         edges.append((u, v, length, width, lon1, lat1, lon2, lat2))
72     if u not in nodes:
73         nodes[u] = {'type': '交叉口'}
74     if v not in nodes:
75         nodes[v] = {'type': '交叉口'}
76     if 'lon' not in nodes[u] or (nodes[u].get('lon', 0) == 0 and lon1 != 0):
77         if lon1 != 0:
78             nodes[u]['lon'] = lon1
79             nodes[u]['lat'] = lat1
80     if 'lon' not in nodes[v] or (nodes[v].get('lon', 0) == 0 and lon2 != 0):
81         if lon2 != 0:
82             nodes[v]['lon'] = lon2
83             nodes[v]['lat'] = lat2
84     return nodes, edges
85 def build_graph(nodes, edges):
86     G = nx.DiGraph()
87     for name, attr in nodes.items():
88         ntype = BUILDING_TYPE.get(name, attr.get('type', '交叉口'))
89         G.add_node(name,
90                     type=ntype,
91                     capacity=CAPACITY.get(name, 0),
92                     lon=str(attr.get('lon', '')),
93                     lat=str(attr.get('lat', '')))
94     added_edges = set()
95     for u, v, length, width, lon1, lat1, lon2, lat2 in edges:
96         cap, vfree = classify_road(width)
97         edge_key_f = (u, v)
98         if edge_key_f not in added_edges:
99             G.add_edge(u, v,
100                        length=length, width=width,
101                        capacity=cap, free_speed=vfree,
102                        free_time=length / vfree,
103                        lon1=lon1, lat1=lat1,
104                        lon2=lon2, lat2=lat2)
105     added_edges.add(edge_key_f)

```

```

106     edge_key_r = (v, u)
107     if edge_key_r not in added_edges:
108         G.add_edge(v, u,
109                     length=length, width=width,
110                     capacity=cap, free_speed=vfree,
111                     free_time=length / vfree,
112                     lon1=lon2, lat1=lat2,
113                     lon2=lon1, lat2=lat1)
114         added_edges.add(edge_key_r)
115     return G
116 def compute_matrices(G, node_order):
117     n = len(node_order)
118     idx = {name: i for i, name in enumerate(node_order)}
119     adj = np.zeros((n, n), dtype=int)
120     dist = np.full((n, n), np.inf)
121     np.fill_diagonal(dist, 0)
122     for u, v, data in G.edges(data=True):
123         i, j = idx[u], idx[v]
124         adj[i][j] = 1
125         d = data['length']
126         if d < dist[i][j]:
127             dist[i][j] = d
128     for k in range(n):
129         for i in range(n):
130             if dist[i][k] == np.inf:
131                 continue
132             for j in range(n):
133                 if dist[k][j] == np.inf:
134                     continue
135                 if dist[i][j] > dist[i][k] + dist[k][j]:
136                     dist[i][j] = dist[i][k] + dist[k][j]
137     return adj, dist
138 def output_node_coords(G, node_order, filepath):
139     with open(filepath, 'w', encoding='utf-8') as f:
140         f.write('node,type,lon,lat\n')
141         for name in node_order:
142             ntype = G.nodes[name]['type']
143             lon = G.nodes[name].get('lon', '')
144             lat = G.nodes[name].get('lat', '')
145             f.write(f'{name},{ntype},{lon},{lat}\n')
146 def main():
147     print("="*60)
148     print("步骤一：校园路网建模")
149     print("="*60)
150     print(f"\n读取数据： {INPUT_FILE}")
151     nodes, edges = parse_network(INPUT_FILE)
152     print(f" 节点数： {len(nodes)}")
153     print(f" 边数（单向）： {len(edges)}")
154     type_count = {}
155     for name, attr in nodes.items():

```

```

156         t = BUILDING_TYPE.get(name, attr.get('type', '交叉口'))
157         type_count[t] = type_count.get(t, 0) + 1
158     print(f" 节点类型分布: {type_count}")
159     G = build_graph(nodes, edges)
160     print(f"\n有向图构建完成:")
161     print(f" 节点数: {G.number_of_nodes()}")
162     print(f" 有向边数: {G.number_of_edges()}")
163     width_dist = {}
164     for u, v, data in G.edges(data=True):
165         w = data['width']
166         width_dist[w] = width_dist.get(w, 0) + 1
167     print(f" 路宽分布: {dict(sorted(width_dist.items()))}")
168     print("\n计算最短距离矩阵...")
169     node_order = list(G.nodes())
170     adj, dist = compute_matrices(G, node_order)
171     gml_path = os.path.join(OUTPUT_DIR, 'campus_graph.gml')
172     nx.write_gml(G, gml_path)
173     print(f"\n输出文件:")
174     print(f" campus_graph.gml")
175     adj_df = pd.DataFrame(adj, index=node_order, columns=node_order)
176     adj_df.to_csv(os.path.join(OUTPUT_DIR, 'adjacency_matrix.csv'))
177     print(f" adjacency_matrix.csv")
178     dist_df = pd.DataFrame(dist, index=node_order, columns=node_order)
179     dist_df.to_csv(os.path.join(OUTPUT_DIR, 'distance_matrix.csv'))
180     print(f" distance_matrix.csv")
181     with open(os.path.join(OUTPUT_DIR, 'node_order.txt'), 'w', encoding='utf-8') as f:
182         for name in node_order:
183             f.write(f"{name}\n")
184     print(f" node_order.txt")
185     output_node_coords(G, node_order, os.path.join(OUTPUT_DIR, 'node_coords.csv'))
186     print(f" node_coords.csv")
187     print(f"\n步骤一完成! 输出目录: {OUTPUT_DIR}")
188 if __name__ == '__main__':
189     main()

```

附录 C：步骤二：OD 需求矩阵生成 (step2_od_matrix.py)

```

1  import pandas as pd
2  import numpy as np
3  import os
4  BASE_DIR = os.path.dirname(os.path.abspath(__file__))
5  STEP1_DIR = os.path.join(BASE_DIR, 'output_step1')
6  OUTPUT_DIR = os.path.join(BASE_DIR, 'output_step2')
7  os.makedirs(OUTPUT_DIR, exist_ok=True)
8  GAMMA = 1.5
9  LOAD_FACTOR = {'morning': 0.90, 'noon': 0.93, 'evening': 0.86}
10 ATTENDANCE = 0.45
11 PERIOD_FLOW = {
12     'morning': {

```

```

13     'sources': ['TYGY', 'XYGY', 'CYGY', 'GYGY'],
14     'targets': ['T_16', 'T_25', 'T_1', 'T_2', 'T_3', 'T_4', 'T_5',
15                 'T_6', 'T_7', 'T_8', 'T_9', 'T_10'],
16 },
17 'noon': {
18     'sources': ['T_16', 'T_25', 'T_1', 'T_2', 'T_3', 'T_4', 'T_5',
19                 'T_6', 'T_7', 'T_8', 'T_9', 'T_10'],
20     'targets': ['TYST', 'XYST', 'GYST', 'SYJ'],
21     'overflow': ['TYGY', 'XYGY', 'CYGY', 'GYGY'],
22 },
23 'evening': {
24     'sources': ['T_16', 'T_25', 'T_1', 'T_2', 'T_3', 'T_4', 'T_5',
25                 'T_6', 'T_7', 'T_8', 'T_9', 'T_10'],
26     'targets': ['TYGY', 'XYGY', 'CYGY', 'GYGY'],
27 },
28 }
29 CAPACITY = {
30     'T_16': 3000, 'T_25': 2000,
31     'T_1': 1000, 'T_2': 1000, 'T_3': 1000, 'T_4': 1000, 'T_5': 1000,
32     'T_6': 1000, 'T_7': 1000, 'T_8': 1000, 'T_9': 1000, 'T_10': 1000,
33     'TYGY': 13000, 'XYGY': 7000, 'CYGY': 3000, 'GYGY': 5000,
34     'TYST': 1100, 'XYST': 1100, 'GYST': 1100,
35     'SYJ': 5000,
36 }
37 MODE_SPLIT = {
38     'short': {'p': 0.70, 'b': 0.25, 'c': 0.05},
39     'medium': {'p': 0.35, 'b': 0.55, 'c': 0.10},
40     'long': {'p': 0.15, 'b': 0.70, 'c': 0.15},
41     'far': {'p': 0.05, 'b': 0.75, 'c': 0.20},
42 }
43 MODES = ['p', 'b', 'c']
44 PERIODS = ['morning', 'noon', 'evening']
45 def get_mode_split(dist):
46     if dist < 500:
47         return MODE_SPLIT['short']
48     elif dist < 1000:
49         return MODE_SPLIT['medium']
50     elif dist < 1500:
51         return MODE_SPLIT['long']
52     else:
53         return MODE_SPLIT['far']
54 def gravity_model(sources, targets, dist_matrix, node_idx, lam, all_nodes):
55     od = pd.DataFrame(0.0, index=all_nodes, columns=all_nodes)
56     for src in sources:
57         if src not in node_idx:
58             continue
59         i = node_idx[src]
60         O_i = CAPACITY.get(src, 1000) * lam
61         denom = 0.0
62         weights = {}

```

```

63     for dst in targets:
64         if dst not in node_idx:
65             continue
66         j = node_idx[dst]
67         d_ij = dist_matrix[i][j]
68         if d_ij <= 0 or d_ij == np.inf or np.isnan(d_ij):
69             d_ij = 10000
70         D_j = CAPACITY.get(dst, 1000)
71         w = D_j * (d_ij ** (-GAMMA))
72         weights[dst] = w
73         denom += w
74     if denom > 0:
75         for dst, w in weights.items():
76             od.loc[src, dst] = O_i * w / denom
77     return od
78 def capacity_allocate(sources, targets, lam, all_nodes, attendance=1.0):
79     od = pd.DataFrame(0.0, index=all_nodes, columns=all_nodes)
80     target_caps = {t: CAPACITY.get(t, 1000) for t in targets if t in all_nodes}
81     total_cap = sum(target_caps.values())
82     for src in sources:
83         if src not in all_nodes:
84             continue
85         O_i = CAPACITY.get(src, 1000) * lam * attendance
86         for dst, cap in target_caps.items():
87             if total_cap > 0 and cap > 0:
88                 od.loc[src, dst] = O_i * cap / total_cap
89     for dst in targets:
90         cap = target_caps.get(dst, 0)
91         if cap <= 0:
92             continue
93         total_to_dst = od.loc[:, dst].sum()
94         if total_to_dst > cap:
95             od.loc[:, dst] *= cap / total_to_dst
96     return od
97 def split_by_mode(od_total, dist_matrix, node_idx):
98     od_modes = {}
99     for mode in MODES:
100         od_modes[mode] = pd.DataFrame(0.0, index=od_total.index, columns=od_total.columns)
101     for src in od_total.index:
102         for dst in od_total.columns:
103             total = od_total.loc[src, dst]
104             if total < 0.01:
105                 continue
106             i = node_idx.get(src, 0)
107             j = node_idx.get(dst, 0)
108             d = 1000
109             if i < dist_matrix.shape[0] and j < dist_matrix.shape[1]:
110                 d = dist_matrix[i][j]
111             if d == np.inf or np.isnan(d):
112                 d = 1000

```

```

113         split = get_mode_split(d)
114         for mode in MODES:
115             od_modes[mode].loc[src, dst] = total * split[mode]
116     return od_modes
117 def main():
118     print("="*60)
119     print("步骤二：OD需求矩阵生成")
120     print("="*60)
121     dist_df = pd.read_csv(os.path.join(STEP1_DIR, 'distance_matrix.csv'), index_col=0)
122     with open(os.path.join(STEP1_DIR, 'node_order.txt'), 'r', encoding='utf-8') as f:
123         node_order = [line.strip() for line in f if line.strip()]
124     node_idx = {name: i for i, name in enumerate(node_order)}
125     dist_matrix = dist_df.values
126     for period in PERIODS:
127         print(f"\n--- {period} ---")
128         flow = PERIOD_FLOW[period]
129         sources = flow['sources']
130         targets = flow['targets']
131         lam = LOAD_FACTOR[period]
132         if period == 'morning':
133             od_total = capacity_allocate(sources, targets, lam, node_order, ATTENDANCE)
134         elif period == 'evening':
135             od_total = gravity_model(sources, targets, dist_matrix, node_idx, lam, node_order)
136         else:
137             od_total = gravity_model(sources, targets, dist_matrix, node_idx, lam, node_order)
138             overflow_targets = flow.get('overflow', [])
139             if overflow_targets:
140                 for dst in targets:
141                     cap = CAPACITY.get(dst, 0)
142                     if cap <= 0:
143                         continue
144                     total_to = od_total.loc[:, dst].sum()
145                     if total_to > cap:
146                         od_total.loc[:, dst] *= cap / total_to
147             for src in sources:
148                 if src not in od_total.index:
149                     continue
150                 used = od_total.loc[src, targets].sum()
151                 O_i = CAPACITY.get(src, 1000) * lam
152                 remaining = max(0, O_i - used)
153                 if remaining <= 0.01:
154                     continue
155                 dorm_caps = {d: CAPACITY.get(d, 1000) for d in overflow_targets if d in
                             node_order}
156                 dorm_total_cap = sum(dorm_caps.values())
157                 for dst, dc in dorm_caps.items():
158                     if dorm_total_cap > 0 and dc > 0:
159                         od_total.loc[src, dst] += remaining * dc / dorm_total_cap
160     od_modes = split_by_mode(od_total, dist_matrix, node_idx)
161     od_bus = pd.DataFrame(0.0, index=node_order, columns=node_order)

```

```

162     od_bus.to_csv(os.path.join(OUTPUT_DIR, f'od_{period}_bus.csv'))
163     total_demand = 0
164     for mode in MODES:
165         od_modes[mode].to_csv(os.path.join(OUTPUT_DIR, f'od_{period}_{mode}.csv'))
166         mode_demand = od_modes[mode].values.sum()
167         total_demand += mode_demand
168         print(f" {mode}: {mode_demand:.0f} 人")
169     od_total.to_csv(os.path.join(OUTPUT_DIR, f'od_{period}_total.csv'))
170     print(f" 总计: {total_demand:.0f} 人")
171     print(f"\n步骤二完成! 输出目录: {OUTPUT_DIR}")
172 if __name__ == '__main__':
173     main()

```

附录 D: 步骤三: 多方式用户均衡分配 (step3_ue_fw.py)

```

1  import networkx as nx
2  import pandas as pd
3  import numpy as np
4  import os, math, heapq
5  from collections import defaultdict
6  BASE_DIR = os.path.dirname(os.path.abspath(__file__))
7  STEP1_DIR = os.path.join(BASE_DIR, 'output_step1')
8  STEP2_DIR = os.path.join(BASE_DIR, 'output_step2')
9  OUTPUT_DIR = os.path.join(BASE_DIR, 'output_step3')
10 os.makedirs(OUTPUT_DIR, exist_ok=True)
11 ALPHA, BETA = 0.15, 4.0
12 XI = {'p': 0.10, 'b': 0.30, 'c': 1.00}
13 V_FREE = {'p': 1.39, 'b': 4.17, 'c': 5.56}
14 OMEGA = {'pb': 0.5, 'pc': 1.5, 'bc': 0.8}
15 MODES = ['p', 'b', 'c']
16 PERIODS = ['morning', 'noon', 'evening']
17 FW_MAX = 20
18 LOS_LEVELS = [(0.35, 'A'), (0.55, 'B'), (0.75, 'C'), (0.90, 'D'), (1.00, 'E')]
19 def bpr_cost(t0, cap, flow):
20     if cap <= 0:
21         return t0 * 10
22     return t0 * (1 + ALPHA * (flow / cap) ** BETA)
23 def dijkstra(source, adj, cost_func):
24     dist = {source: 0.0}
25     prev = {}
26     heap = [(0.0, source)]
27     while heap:
28         d, u = heapq.heappop(heap)
29         if d > dist.get(u, math.inf):
30             continue
31         for v, eid in adj.get(u, []):
32             nd = d + cost_func(eid)
33             if nd < dist.get(v, math.inf):
34                 dist[v] = nd

```



```

35         prev[v] = u
36         heapq.heappush(heap, (nd, v))
37     return dist, prev
38 def get_path(prev, src, dst):
39     if dst not in prev:
40         return None
41     path = [dst]
42     cur = dst
43     while cur != src:
44         cur = prev[cur]
45         path.append(cur)
46     return path[::-1]
47 def assign_aon(od, adj, eattr, eid_map, x_eq):
48     aon_mode = defaultdict(float)
49     for (src, dst, mode), demand in od.items():
50         if demand <= 0 or src not in adj or dst not in adj:
51             continue
52         def cf(eid, _mode=mode):
53             a = eattr[eid]
54             return bpr_cost(a['t0'][_mode], a['capacity'], x_eq.get(eid, 0))
55         dist, prev = dijkstra(src, adj, cf)
56         path = get_path(prev, src, dst)
57         if path is None:
58             continue
59         for i in range(len(path) - 1):
60             eid = eid_map.get((path[i], path[i + 1]))
61             if eid is not None:
62                 aon_mode[(eid, mode)] += demand
63     aon_eq = defaultdict(float)
64     for (eid, mode), flow in aon_mode.items():
65         aon_eq[eid] += XI[mode] * flow
66     return dict(aon_eq), dict(aon_mode)
67 def frank_wolfe(od, adj, eattr, eid_map):
68     x_eq_zero = {eid: 0.0 for eid in eattr}
69     aux_eq0, aux_mode0 = assign_aon(od, adj, eattr, eid_map, x_eq_zero)
70     x_mode = defaultdict(float, aux_mode0)
71     x_eq = defaultdict(float, aux_eq0)
72     prev_obj = math.inf
73     for it in range(FW_MAX):
74         obj = 0.0
75         for eid in eattr:
76             a = eattr[eid]
77             obj += bpr_cost(a['t0']['p'], a['capacity'], x_eq.get(eid, 0))
78     y_eq, y_mode = assign_aon(od, adj, eattr, eid_map, x_eq)
79     lo, hi = 0.0, 1.0
80     for _ in range(20):
81         lam = (lo + hi) / 2
82         grad = 0.0
83         for eid in eattr:
84             xe, ye = x_eq.get(eid, 0), y_eq.get(eid, 0)

```

```

85         if abs(ye - xe) < 1e-10:
86             continue
87         xnew = max(xe + lam * (ye - xe), 0)
88         a = eattr[eid]
89         grad += bpr_cost(a['t0']['p'], a['capacity'], xnew) * (ye - xe)
90     if grad > 0:
91         hi = lam
92     else:
93         lo = lam
94     lam_opt = (lo + hi) / 2
95     new_x_mode = defaultdict(float)
96     for k in set(x_mode) | set(y_mode):
97         v = x_mode.get(k, 0) + lam_opt * (y_mode.get(k, 0) - x_mode.get(k, 0))
98         if v > 0:
99             new_x_mode[k] = v
100     new_x_eq = defaultdict(float)
101     for eid in eattr:
102         v = x_eq.get(eid, 0) + lam_opt * (y_eq.get(eid, 0) - x_eq.get(eid, 0))
103         if v > 0:
104             new_x_eq[eid] = v
105     x_mode, x_eq = new_x_mode, new_x_eq
106     rel = abs(obj - prev_obj) / (abs(prev_obj) + 1e-10)
107     if rel < 1e-4 and it > 5:
108         break
109     prev_obj = obj
110     return dict(x_mode), dict(x_eq)
111 def build_adj_eattr(G):
112     adj = defaultdict(list)
113     eid_map = {}
114     eattr = {}
115     for eid, (u, v, data) in enumerate(G.edges(data=True)):
116         L = float(data['length'])
117         cap = float(data['capacity'])
118         eattr[eid] = {
119             'u': u, 'v': v, 'length': L, 'capacity': cap,
120             'width': float(data.get('width', 6)),
121             't0': {m: L / V_FREE[m] for m in MODES}
122         }
123         adj[u].append((v, eid))
124         eid_map[(u, v)] = eid
125     return adj, eid_map, eattr
126 def load_od(period):
127     od = {}
128     for mode in MODES:
129         fp = os.path.join(STEP2_DIR, f'od_{period}_{mode}.csv')
130         if not os.path.exists(fp):
131             continue
132         df = pd.read_csv(fp, index_col=0)
133         for src in df.index:
134             for dst in df.columns:

```

```

135         v = float(df.loc[src, dst])
136         if v > 0.01:
137             od[(src, dst, mode)] = v
138     return od
139 def get_los(vc):
140     if vc <= 0.35: return 'A'
141     if vc <= 0.55: return 'B'
142     if vc <= 0.75: return 'C'
143     if vc <= 0.90: return 'D'
144     if vc <= 1.00: return 'E'
145     return 'F'
146 def compute_metrics(x_mode, x_eq, eattr, eid_map):
147     ttt = 0.0
148     for (eid, mode), flow in x_mode.items():
149         a = eattr[eid]
150         travel_time = bpr_cost(a['t0'][mode], a['capacity'], x_eq.get(eid, 0))
151         ttt += flow * travel_time
152     flow_by_mode = defaultdict(lambda: {'p': 0, 'b': 0, 'c': 0})
153     for (eid, mode), flow in x_mode.items():
154         flow_by_mode[eid][mode] = flow
155     S = 0.0
156     for eid, flows in flow_by_mode.items():
157         a = eattr[eid]
158         width = a['width'] if a['width'] < 99 else 6.0
159         fp, fb, fc = flows['p'], flows['b'], flows['c']
160         risk = (fp * fb * OMEGA['pb'] + fp * fc * OMEGA['pc'] + fb * fc * OMEGA['bc']) / width
161         S += risk
162     return ttt, S
163 def main():
164     print("=" * 60)
165     print("步骤三：多类用户均衡交通分配")
166     print("=" * 60)
167     G = nx.read_gml(os.path.join(STEP1_DIR, 'campus_graph.gml'))
168     adj, eid_map, eattr = build_adj_eattr(G)
169     print(f"图: {G.number_of_nodes()} 节点, {G.number_of_edges()} 有向边, {len(eattr)} 路段")
170     baseline_results = {}
171     for period in PERIODS:
172         print(f"\n--- {period} ---")
173         od = load_od(period)
174         print(f"OD需求: {len(od)} 条")
175         x_mode, x_eq = frank_wolfe(od, adj, eattr, eid_map)
176         ttt, S = compute_metrics(x_mode, x_eq, eattr, eid_map)
177         flow_by_mode_link = defaultdict(lambda: {'p': 0, 'b': 0, 'c': 0})
178         for (eid, mode), flow in x_mode.items():
179             flow_by_mode_link[eid][mode] = flow
180         link_records = []
181         high_vc_count = 0
182         max_vc = 0
183         for eid, a in eattr.items():
184             eq_flow = x_eq.get(eid, 0)

```

```

185     vc = eq_flow / a['capacity'] if a['capacity'] > 0 else 0
186     if vc > max_vc:
187         max_vc = vc
188     if vc > 0.75:
189         high_vc_count += 1
190     fm = flow_by_mode_link.get(eid, {'p':0,'b':0,'c':0})
191     width_r = a['width'] if a['width'] < 99 else 6.0
192     risk = (fm['p']*fm['b']*OMEGA['pb'] + fm['p']*fm['c']*OMEGA['pc'] + fm['b']*fm['c']*
193             OMEGA['bc']) / width_r
194     link_records.append({
195         'edge_id': eid,
196         'from': a['u'], 'to': a['v'],
197         'length_m': f'{a["length"]:.0f}', 'width_m': f'{a["width"]:.1f}',
198         'capacity': f'{a["capacity"]:.1f}',
199         'flow_p': f'{fm["p"]:.1f}', 'flow_b': f'{fm["b"]:.1f}', 'flow_c': f'{fm["c"]:.1f}'
200         ,
201         'flow_eq': f'{eq_flow:.1f}',
202         'saturation': f'{vc:.4f}', 'LOS': get_los(vc),
203         'risk': f'{risk:.1f}',
204     })
205     link_df = pd.DataFrame(link_records)
206     link_df.to_csv(os.path.join(OUTPUT_DIR, f'link_flow_{period}.csv'), index=False)
207     print(f" TTT: {ttt:,.0f} 人·秒")
208     print(f" S: {S:,.0f}")
209     print(f" V/C>0.75: {high_vc_count} 条")
210     print(f" max V/C: {max_vc:.3f}")
211     baseline_results[period] = {'TTT': ttt, 'S': S, 'high_vc': high_vc_count, 'max_vc':
212                                max_vc}
213     with open(os.path.join(OUTPUT_DIR, 'baseline.txt'), 'w', encoding='utf-8') as f:
214         f.write("period,TTT,S\n")
215         for period in ['morning', 'noon', 'evening']:
216             if period in baseline_results:
217                 f.write(f"{period},{baseline_results[period]['TTT']:.1f},{baseline_results[period]
218                        }['S']:.1f\n")
219     with open(os.path.join(OUTPUT_DIR, 'baseline_readable.txt'), 'w', encoding='utf-8') as f:
220         f.write("基准现状评价结果\n")
221         f.write("=" * 50 + "\n\n")
222         period_names = {'morning': '早高峰', 'noon': '午高峰', 'evening': '晚高峰'}
223         for period, res in baseline_results.items():
224             f.write(f"{period_names[period]} ({period}):\n")
225             f.write(f" TTT (总行程时间): {res['TTT']:.0f} 人·秒\n")
226             f.write(f" S (冲突风险): {res['S']:.0f}\n")
227             f.write(f" V/C>0.75路段: {res['high_vc']} 条\n")
228             f.write(f" max V/C: {res['max_vc']:.3f}\n\n")
229         total_ttt = sum(r['TTT'] for r in baseline_results.values())
230         total_s = sum(r['S'] for r in baseline_results.values())
231         f.write(f"三时段合计:\n")
232         f.write(f" TTT合计: {total_ttt:,.0f} 人·秒\n")
233         f.write(f" S合计: {total_s:,.0f}\n")
234     print(f"\n步骤三完成! 输出目录: {OUTPUT_DIR}")

```

```

231 if __name__ == '__main__':
232     main()

```

附录 E：步骤四：交通组织方案设计与对比 (step4_scenarios.py)

```

1  import networkx as nx
2  import pandas as pd
3  import numpy as np
4  import os, math, heapq
5  from collections import defaultdict
6  import copy
7  BASE_DIR = os.path.dirname(os.path.abspath(__file__))
8  STEP1_DIR = os.path.join(BASE_DIR, 'output_step1')
9  STEP2_DIR = os.path.join(BASE_DIR, 'output_step2')
10 STEP3_DIR = os.path.join(BASE_DIR, 'output_step3')
11 OUTPUT_DIR = os.path.join(BASE_DIR, 'output_step4')
12 os.makedirs(OUTPUT_DIR, exist_ok=True)
13 ALPHA, BETA = 0.15, 4.0
14 XI = {'p': 0.10, 'b': 0.30, 'c': 1.00}
15 V_FREE = {'p': 1.39, 'b': 4.17, 'c': 5.56}
16 OMEGA = {'pb': 0.5, 'pc': 1.5, 'bc': 0.8}
17 MODES = ['p', 'b', 'c']
18 PERIODS = ['morning', 'noon', 'evening']
19 FW_MAX = 20
20 ONEWAY_A = {
21     'morning': [('N4', 'N14'), ('N30', 'N16'), ('N46', 'N49')],
22 }
23 RESTRICT_B = {
24     'noon': [
25         ('N16', 'N30'),
26         ('N48', 'N49'),
27         ('N25', 'N26'),
28         ('N26', 'N27'),
29         ('N14', 'N15'), ('N14', 'N4'),
30         ('N30', 'N12'), ('N28', 'N12'),
31         ('N28', 'N29'), ('N29', 'N31'),
32         ('N12', 'N13'),
33     ],
34 }
35 ISOLATE_C = [
36     ('N4', 'N14'), ('N14', 'N15'), ('N15', 'N16'), ('N30', 'N16'),
37     ('N28', 'N29'), ('N28', 'N31'), ('N33', 'N34'), ('N33', 'N46'),
38     ('N46', 'N49'), ('N49', 'N48'), ('N37', 'N47'), ('N48', 'N38'),
39     ('N33', 'N61'), ('N61', 'N62'),
40 ]
41 ISO_SET = set()
42 for u, v in ISOLATE_C:
43     ISO_SET.add((u, v))
44     ISO_SET.add((v, u))

```

```

45 def bpr_cost(t0, cap, flow):
46     if cap <= 0: return t0 * 10
47     return t0 * (1 + ALPHA * (flow / cap) ** BETA)
48 def dijkstra(source, adj, cost_func):
49     dist = {source: 0.0}; prev = {}; heap = [(0.0, source)]
50     while heap:
51         d, u = heapq.heappop(heap)
52         if d > dist.get(u, math.inf): continue
53         for v, eid in adj.get(u, []):
54             nd = d + cost_func(eid)
55             if nd < dist.get(v, math.inf):
56                 dist[v] = nd; prev[v] = u; heapq.heappush(heap, (nd, v))
57     return dist, prev
58 def get_path(prev, src, dst):
59     if dst not in prev: return None
60     path = [dst]; cur = dst
61     while cur != src: cur = prev[cur]; path.append(cur)
62     return path[::-1]
63 def assign_aon(od, adj, eattr, eid_map, x_eq):
64     aon_mode = defaultdict(float)
65     for (src, dst, mode), demand in od.items():
66         if demand <= 0 or src not in adj or dst not in adj: continue
67         def cf(eid, _mode=mode):
68             a = eattr[eid]; return bpr_cost(a['t0'][_mode], a['capacity'], x_eq.get(eid, 0))
69         dist, prev = dijkstra(src, adj, cf)
70         path = get_path(prev, src, dst)
71         if path is None: continue
72         for i in range(len(path) - 1):
73             eid = eid_map.get((path[i], path[i + 1]))
74             if eid is not None: aon_mode[(eid, mode)] += demand
75     aon_eq = defaultdict(float)
76     for (eid, mode), flow in aon_mode.items(): aon_eq[eid] += XI[mode] * flow
77     return dict(aon_eq), dict(aon_mode)
78 def frank_wolfe(od, adj, eattr, eid_map):
79     x_eq_zero = {eid: 0.0 for eid in eattr}
80     aux_eq0, aux_mode0 = assign_aon(od, adj, eattr, eid_map, x_eq_zero)
81     x_mode = defaultdict(float, aux_mode0); x_eq = defaultdict(float, aux_eq0)
82     prev_obj = math.inf
83     for it in range(FW_MAX):
84         obj = sum(bpr_cost(eattr[eid]['t0']['p'], eattr[eid]['capacity'], x_eq.get(eid, 0)) for
85                   eid in eattr)
86         y_eq, y_mode = assign_aon(od, adj, eattr, eid_map, x_eq)
87         lo, hi = 0.0, 1.0
88         for _ in range(20):
89             lam = (lo + hi) / 2; grad = 0.0
90             for eid in eattr:
91                 xe, ye = x_eq.get(eid, 0), y_eq.get(eid, 0)
92                 if abs(ye - xe) < 1e-10: continue
93                 xnew = max(xe + lam * (ye - xe), 0); a = eattr[eid]
94                 grad += bpr_cost(a['t0']['p'], a['capacity'], xnew) * (ye - xe)

```

```

94         if grad > 0: hi = lam
95         else: lo = lam
96     lam_opt = (lo + hi) / 2
97     new_x_mode = defaultdict(float)
98     for k in set(x_mode) | set(y_mode):
99         v = x_mode.get(k, 0) + lam_opt * (y_mode.get(k, 0) - x_mode.get(k, 0))
100         if v > 0: new_x_mode[k] = v
101     new_x_eq = defaultdict(float)
102     for eid in eattr:
103         v = x_eq.get(eid, 0) + lam_opt * (y_eq.get(eid, 0) - x_eq.get(eid, 0))
104         if v > 0: new_x_eq[eid] = v
105     x_mode, x_eq = new_x_mode, new_x_eq
106     rel = abs(obj - prev_obj) / (abs(prev_obj) + 1e-10)
107     if rel < 1e-4 and it > 5: break
108     prev_obj = obj
109     return dict(x_mode), dict(x_eq)
110 def build_adj_eattr(G):
111     adj = defaultdict(list); eid_map = {}; eattr = {}
112     for eid, (u, v, data) in enumerate(G.edges(data=True)):
113         L = float(data['length']); cap = float(data['capacity'])
114         eattr[eid] = {'u': u, 'v': v, 'length': L, 'capacity': cap,
115                     'width': float(data.get('width', 6)),
116                     't0': {m: L / V_FREE[m] for m in MODES}}
117         adj[u].append((v, eid)); eid_map[(u, v)] = eid
118     return adj, eid_map, eattr
119 def load_od(period):
120     od = {}
121     for mode in MODES:
122         fp = os.path.join(STEP2_DIR, f'od_{period}_{mode}.csv')
123         if not os.path.exists(fp): continue
124         df = pd.read_csv(fp, index_col=0)
125         for src in df.index:
126             for dst in df.columns:
127                 v = float(df.loc[src, dst])
128                 if v > 0.01: od[(src, dst, mode)] = v
129     return od
130 def apply_scenario(G, scenario, period):
131     G_mod = G.copy()
132     if scenario in ('A', 'C'):
133         oneway_edges = ONEWAY_A.get(period, [])
134         for u, v in oneway_edges:
135             if G_mod.has_edge(v, u):
136                 G_mod.remove_edge(v, u)
137     if scenario in ('B', 'C'):
138         restrict_edges = RESTRICT_B.get(period, [])
139         for u, v in restrict_edges:
140             if G_mod.has_edge(u, v):
141                 G_mod[u][v]['capacity'] = float(G_mod[u][v]['capacity']) * 0.6
142     return G_mod
143 def compute_metrics(x_mode, x_eq, eattr):

```

```

144     ttt = 0.0
145     for (eid, mode), flow in x_mode.items():
146         a = eattr[eid]
147         travel_time = bpr_cost(a['t0'][mode], a['capacity'], x_eq.get(eid, 0))
148         ttt += flow * travel_time
149     flow_by_mode = defaultdict(lambda: {'p': 0, 'b': 0, 'c': 0})
150     for (eid, mode), flow in x_mode.items():
151         flow_by_mode[eid][mode] = flow
152     S = 0.0
153     for eid, flows in flow_by_mode.items():
154         a = eattr[eid]
155         width = a['width'] if a['width'] < 99 else 6.0
156         fp, fb, fc = flows['p'], flows['b'], flows['c']
157         risk = (fp * fb * OMEGA['pb'] + fp * fc * OMEGA['pc'] + fb * fc * OMEGA['bc']) / width
158         u, v = a['u'], a['v']
159         if scenario_is_isolated(u, v, current_scenario):
160             risk *= 0.4
161         S += risk
162     return ttt, S
163 current_scenario = 'baseline'
164 def scenario_is_isolated(u, v, scenario):
165     if scenario != 'C':
166         return False
167     return (u, v) in ISO_SET
168 def run_scenario(scenario_name, G_base):
169     global current_scenario
170     current_scenario = scenario_name
171     results = {}
172     for period in PERIODS:
173         G_mod = apply_scenario(G_base, scenario_name, period)
174         adj, eid_map, eattr = build_adj_eattr(G_mod)
175         od = load_od(period)
176         if scenario_name in ('B', 'C') and period == 'noon':
177             new_od = {}
178             for (s, d, m), val in od.items():
179                 new_od[(s, d, m)] = val * 0.3 if m == 'c' else val
180             od = new_od
181         x_mode, x_eq = frank_wolfe(od, adj, eattr, eid_map)
182         ttt, S = compute_metrics(x_mode, x_eq, eattr)
183         vc_values = []
184         for eid, a in eattr.items():
185             eq_flow = x_eq.get(eid, 0)
186             vc = eq_flow / a['capacity'] if a['capacity'] > 0 else 0
187             vc_values.append(vc)
188         results[period] = {
189             'TTT': ttt, 'S': S,
190             'avg_vc': np.mean(vc_values) if vc_values else 0,
191             'max_vc': max(vc_values) if vc_values else 0,
192         }
193     return results

```



```

194 def main():
195     print("=" * 60)
196     print("步骤四：情景方案设计与对比")
197     print("=" * 60)
198     G_base = nx.read_gml(os.path.join(STEP1_DIR, 'campus_graph.gml'))
199     scenarios = [
200         ('baseline', '基准（现状）'),
201         ('A', '方案A：关键路段单行线'),
202         ('B', '方案B：午高峰食堂区限行'),
203         ('C', '方案C：综合优化'),
204     ]
205     all_results = {}
206     for sname, sdesc in scenarios:
207         print(f"\n--- {sdesc} ---")
208         res = run_scenario(sname, G_base)
209         all_results[sname] = res
210         for period, r in res.items():
211             print(f" {period}: TTT={r['TTT']:.0f}, S={r['S']:.0f}, avgVC={r['avg_vc']:.4f}")
212     base = all_results['baseline']
213     comparison = []
214     period_names = {'morning': '早高峰', 'noon': '午高峰', 'evening': '晚高峰'}
215     for sname, sdesc in scenarios:
216         row = {'方案': sdesc}
217         total_ttt = sum(all_results[sname][p]['TTT'] for p in PERIODS)
218         total_s = sum(all_results[sname][p]['S'] for p in PERIODS)
219         avg_vc = np.mean([all_results[sname][p]['avg_vc'] for p in PERIODS])
220         max_vc = max(all_results[sname][p]['max_vc'] for p in PERIODS)
221         row['TTT合计'] = round(total_ttt, 0)
222         row['S合计'] = round(total_s, 0)
223         row['avgV/C'] = round(avg_vc, 4)
224         row['maxV/C'] = round(max_vc, 4)
225         base_ttt = sum(base[p]['TTT'] for p in PERIODS)
226         base_s = sum(base[p]['S'] for p in PERIODS)
227         base_avg_vc = np.mean([base[p]['avg_vc'] for p in PERIODS])
228         base_max_vc = max(base[p]['max_vc'] for p in PERIODS)
229         row['eta_TTT'] = round((base_ttt - total_ttt) / base_ttt * 100, 1) if base_ttt > 0 else 0
230         row['eta_S'] = round((base_s - total_s) / base_s * 100, 1) if base_s > 0 else 0
231         row['eta_VC'] = round((base_avg_vc - avg_vc) / base_avg_vc * 100, 1) if base_avg_vc > 0
232             else 0
233         comparison.append(row)
234     comp_df = pd.DataFrame(comparison)
235     comp_df.to_csv(os.path.join(OUTPUT_DIR, 'scenario_comparison.csv'), index=False, encoding='
236         utf-8-sig')
237     print("\n" + "=" * 60)
238     print("方案对比汇总:")
239     print(comp_df.to_string(index=False))
240     print(f"\n输出: {OUTPUT_DIR}/scenario_comparison.csv")
241     print("步骤四完成!")
242 if __name__ == '__main__':
243     main()

```

附录 F：步骤五：MinMax V/C 增量分流 (step5_k_shortest.py)

```
1 import os
2 import math
3 import heapq
4 from collections import defaultdict
5 import networkx as nx
6 import pandas as pd
7 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
8 STEP1_DIR = os.path.join(BASE_DIR, 'output_step1')
9 STEP2_DIR = os.path.join(BASE_DIR, 'output_step2')
10 STEP3_DIR = os.path.join(BASE_DIR, 'output_step3')
11 OUTPUT_DIR = os.path.join(BASE_DIR, 'output_step5_minmax')
12 os.makedirs(OUTPUT_DIR, exist_ok=True)
13 MODES = ['p', 'b', 'c']
14 PERIODS = ['morning', 'noon', 'evening']
15 PERIOD_NAMES = {'morning': '早高峰', 'noon': '午高峰', 'evening': '晚高峰'}
16 XI = {'p': 0.10, 'b': 0.30, 'c': 1.00}
17 V_FREE = {'p': 1.39, 'b': 4.17, 'c': 5.56}
18 BATCHES = 60
19 LAMBDA_CONGESTION = 65.0
20 POWER_CONGESTION = 4.0
21 OVER_CAP_PENALTY = 140.0
22 NARROW_PENALTY = 0.40
23 def build_edge_data(G):
24     adj = defaultdict(list)
25     edge_data = {}
26     for eid, (u, v, data) in enumerate(G.edges(data=True)):
27         length = float(data.get('length', 100))
28         width = float(data.get('width', 6))
29         capacity = float(data.get('capacity', 600))
30         edge_data[eid] = {
31             'u': u,
32             'v': v,
33             'length': length,
34             'width': width,
35             'capacity': capacity,
36             'is_virtual': width >= 99,
37         }
38         adj[u].append((v, eid))
39     return adj, edge_data
40 def edge_cost(eid, mode, edge_data, flow_eq):
41     e = edge_data[eid]
42     cap = max(e['capacity'], 1.0)
43     vc = flow_eq.get(eid, 0.0) / cap
44     base_time = e['length'] / V_FREE[mode]
45     congestion = LAMBDA_CONGESTION * (vc ** POWER_CONGESTION)
46     over_cap = OVER_CAP_PENALTY * (max(0.0, vc - 0.75) ** 2)
```

```

47     narrow = 0.0
48     if not e['is_virtual']:
49         narrow = NARROW_PENALTY * max(0.0, (8.0 - e['width']) / 8.0)
50     return base_time * (1.0 + congestion + over_cap + narrow)
51 def dijkstra_path(src, dst, adj, edge_data, flow_eq, mode):
52     dist = {src: 0.0}
53     prev = {}
54     heap = [(0.0, src)]
55     while heap:
56         d, u = heapq.heappop(heap)
57         if d > dist.get(u, math.inf):
58             continue
59         if u == dst:
60             break
61         for v, eid in adj.get(u, []):
62             nd = d + edge_cost(eid, mode, edge_data, flow_eq)
63             if nd < dist.get(v, math.inf):
64                 dist[v] = nd
65                 prev[v] = (u, eid)
66                 heapq.heappush(heap, (nd, v))
67     if dst not in prev:
68         return None, math.inf
69     nodes = [dst]
70     eids = []
71     cur = dst
72     while cur != src:
73         p, eid = prev[cur]
74         eids.append(eid)
75         cur = p
76         nodes.append(cur)
77     nodes.reverse()
78     eids.reverse()
79     return (nodes, eids), dist[dst]
80 def load_od(period):
81     od = []
82     for mode in MODES:
83         fp = os.path.join(STEP2_DIR, f'od_{period}_{mode}.csv')
84         if not os.path.exists(fp):
85             continue
86         df = pd.read_csv(fp, index_col=0)
87         for src in df.index:
88             for dst in df.columns:
89                 demand = float(df.loc[src, dst])
90                 if demand > 0.01:
91                     od.append({'source': src, 'target': dst, 'mode': mode, 'demand': demand})
92     od.sort(key=lambda x: x['demand'], reverse=True)
93     return od
94 def get_los(vc):
95     if vc <= 0.35:
96         return 'A'

```

```

97     if vc <= 0.55:
98         return 'B'
99     if vc <= 0.75:
100         return 'C'
101     if vc <= 0.90:
102         return 'D'
103     if vc <= 1.00:
104         return 'E'
105     return 'F'
106 def load_baseline(period):
107     fp = os.path.join(STEP3_DIR, f'link_flow_{period}.csv')
108     if not os.path.exists(fp):
109         return {}
110     df = pd.read_csv(fp)
111     return {(r['from'], r['to']): float(r['saturation']) for _, r in df.iterrows()}
112 def optimize_period(period, G, adj, edge_data):
113     od = load_od(period)
114     flow_eq = defaultdict(float)
115     flow_mode = defaultdict(float)
116     path_counter = defaultdict(float)
117     print(f"\n--- {PERIOD_NAMES[period]} ---")
118     print(f" OD记录: {len(od)} 条, 总需求: {sum(x['demand'] for x in od):.0f} 人")
119     for batch in range(BATCHES):
120         for item in od:
121             src, dst, mode = item['source'], item['target'], item['mode']
122             if src not in G.nodes or dst not in G.nodes:
123                 continue
124             batch_flow = item['demand'] / BATCHES
125             result, _ = dijkstra_path(src, dst, adj, edge_data, flow_eq, mode)
126             if result is None:
127                 continue
128             nodes, eids = result
129             eq_flow = batch_flow * XI[mode]
130             for eid in eids:
131                 flow_eq[eid] += eq_flow
132                 flow_mode[(eid, mode)] += batch_flow
133             path_counter[(period, src, dst, mode, ' -> '.join(nodes))] += batch_flow
134 link_records = []
135 opt_vc = {}
136 for eid, e in edge_data.items():
137     flow = flow_eq.get(eid, 0.0)
138     cap = e['capacity']
139     vc = flow / cap if cap > 0 else 0.0
140     opt_vc[(e['u'], e['v'])] = vc
141     link_records.append({
142         'u': e['u'],
143         'v': e['v'],
144         'length': e['length'],
145         'width': e['width'],
146         'capacity': cap,

```

```

147     'flow_pcu': round(flow, 2),
148     'flow_p': round(flow_mode.get((eid, 'p'), 0.0), 2),
149     'flow_b': round(flow_mode.get((eid, 'b'), 0.0), 2),
150     'flow_c': round(flow_mode.get((eid, 'c'), 0.0), 2),
151     'V/C': round(vc, 4),
152     'LOS': get_los(vc),
153 })
154 link_df = pd.DataFrame(link_records)
155 link_df.to_csv(os.path.join(OUTPUT_DIR, f'link_flow_optimized_{period}.csv'), index=False,
156               encoding='utf-8-sig')
157 base_vc = load_baseline(period)
158 compare_records = []
159 for eid, e in edge_data.items():
160     edge = (e['u'], e['v'])
161     b = base_vc.get(edge, 0.0)
162     o = opt_vc.get(edge, 0.0)
163     compare_records.append({
164         'u': e['u'],
165         'v': e['v'],
166         'width': e['width'],
167         'capacity': e['capacity'],
168         'baseline_vc': round(b, 4),
169         'optimized_vc': round(o, 4),
170         'delta_vc': round(o - b, 4),
171         'reduction_pct': round((b - o) / b * 100, 2) if b > 1e-9 else 0.0,
172     })
173 compare_df = pd.DataFrame(compare_records)
174 compare_df.to_csv(os.path.join(OUTPUT_DIR, f'vc_compare_{period}.csv'), index=False,
175                  encoding='utf-8-sig')
176 path_rows = []
177 for (p, src, dst, mode, route), flow in path_counter.items():
178     path_rows.append({
179         'period': PERIOD_NAMES[p],
180         'source': src,
181         'target': dst,
182         'mode': mode,
183         'route': route,
184         'flow_persons': round(flow, 2),
185     })
186 real_links = link_df[link_df['width'] < 99]
187 high = int((real_links['V/C'] > 0.75).sum())
188 max_vc = float(real_links['V/C'].max()) if len(real_links) else 0.0
189 mean_pos = float(real_links[real_links['V/C'] > 0]['V/C'].mean()) if (real_links['V/C'] > 0)
190 .any() else 0.0
191 base_real = compare_df[compare_df['width'] < 99]
192 base_max = float(base_real['baseline_vc'].max()) if len(base_real) else 0.0
193 base_high = int((base_real['baseline_vc'] > 0.75).sum())
194 print(f" 基准: max V/C={base_max:.3f}, V/C>0.75={base_high} 条")
195 print(f" 优化: max V/C={max_vc:.3f}, V/C>0.75={high} 条, 正流均值={mean_pos:.3f}")
196 return {

```

```

194     'period': period,
195     'base_max': base_max,
196     'opt_max': max_vc,
197     'base_high': base_high,
198     'opt_high': high,
199     'mean_pos': mean_pos,
200     'path_rows': path_rows,
201 }
202 def main():
203     print("=" * 60)
204     print("步骤五: MinMax V/C 系统优化分配")
205     print("=" * 60)
206     G = nx.read_gml(os.path.join(STEP1_DIR, 'campus_graph.gml'))
207     adj, edge_data = build_edge_data(G)
208     print(f"图: {G.number_of_nodes()} 节点, {G.number_of_edges()} 有向边")
209     summaries = []
210     all_path_rows = []
211     for period in PERIODS:
212         res = optimize_period(period, G, adj, edge_data)
213         summaries.append({k: v for k, v in res.items() if k != 'path_rows'})
214         all_path_rows.extend(res['path_rows'])
215     pd.DataFrame(all_path_rows).to_csv(os.path.join(OUTPUT_DIR, 'path_flow_optimized.csv'),
216                                       index=False, encoding='utf-8-sig')
217     pd.DataFrame(summaries).to_csv(os.path.join(OUTPUT_DIR, 'summary.csv'), index=False,
218                                     encoding='utf-8-sig')
219     print(f"\n步骤五完成! 输出目录: {OUTPUT_DIR}")
220 if __name__ == '__main__':
221     main()

```

附录 G: 步骤六: 方案评价与报告输出 (step6_evaluate.py)

```

1  import pandas as pd
2  import os
3  BASE_DIR = os.path.dirname(os.path.abspath(__file__))
4  STEP3_DIR = os.path.join(BASE_DIR, 'output_step3')
5  STEP4_DIR = os.path.join(BASE_DIR, 'output_step4')
6  OUTPUT_DIR = os.path.join(BASE_DIR, 'output_step6')
7  os.makedirs(OUTPUT_DIR, exist_ok=True)
8  PERIODS = ['morning', 'noon', 'evening']
9  PERIOD_NAMES = {'morning': '早高峰', 'noon': '午高峰', 'evening': '晚高峰'}
10 def main():
11     print("=" * 60)
12     print("步骤六: 效果评价")
13     print("=" * 60)
14     comp_path = os.path.join(STEP4_DIR, 'scenario_comparison.csv')
15     comp_df = pd.read_csv(comp_path)
16     print(f"\n方案对比表:\n{comp_df.to_string(index=False)}")
17     best_idx = comp_df['eta_S'].idxmax()
18     best_scenario = comp_df.loc[best_idx]

```

```

19 best_name = best_scenario['方案']
20 print(f"\n最优方案: {best_name}")
21 print(f" TTT改善: {best_scenario['eta_TTT']:.1f}%")
22 print(f" S改善: {best_scenario['eta_S']:.1f}%")
23 print(f" V/C改善: {best_scenario['eta_VC']:.1f}%")
24 los_dist = {}
25 for period in PERIODS:
26     fp = os.path.join(STEP3_DIR, f'link_flow_{period}.csv')
27     if not os.path.exists(fp):
28         continue
29     df = pd.read_csv(fp)
30     los_counts = df['LOS'].value_counts().to_dict()
31     los_dist[PERIOD_NAMES[period]] = los_counts
32 report = []
33 report.append("=" * 60)
34 report.append("浙师大校园微交通系统综合治理 — 效果评价报告")
35 report.append("=" * 60)
36 report.append("\n一、 方案对比结果")
37 report.append("-" * 40)
38 for _, row in comp_df.iterrows():
39     report.append(f"\n {row['方案']}:")
40     report.append(f" TTT合计: {row['TTT合计']:, .0f} 人·秒 (改善 {row['eta_TTT']:.1f}%)")
41     report.append(f" S合计: {row['S合计']:, .0f} (改善 {row['eta_S']:.1f}%)")
42     report.append(f" avgV/C: {row['avgV/C']:.4f} (改善 {row['eta_VC']:.1f}%)")
43     report.append(f" maxV/C: {row['maxV/C']:.4f}")
44 report.append(f"\n二、 最优方案: {best_name}")
45 report.append("-" * 40)
46 report.append(f" 以冲突风险(S)改善率作为主要评价指标:")
47 report.append(f" TTT总行程时间改善: {best_scenario['eta_TTT']:.1f}%")
48 report.append(f" 人车冲突风险降低: {best_scenario['eta_S']:.1f}%")
49 report.append(f" 平均V/C改善: {best_scenario['eta_VC']:.1f}%")
50 report.append("\n三、 基准LOS分布")
51 report.append("-" * 40)
52 for period_name, los_counts in los_dist.items():
53     report.append(f"\n {period_name}:")
54     for los_level in ['A', 'B', 'C', 'D', 'E', 'F']:
55         count = los_counts.get(los_level, 0)
56         if count > 0:
57             report.append(f" LOS {los_level}: {count} 条路段")
58 report.append("\n四、 结论")
59 report.append("-" * 40)
60 report.append(f" 推荐方案: {best_name}")
61 if best_scenario['eta_S'] > 10:
62     report.append(" 该方案显著降低了人车冲突风险, ")
63     report.append(" 在保障校园交通安全方面效果突出。")
64 report.append(f" 总行程时间基本保持稳定 (变化 {best_scenario['eta_TTT']:.1f}%) , ")
65 report.append(" 说明优化措施未牺牲通行效率。")
66 report_text = '\n'.join(report)
67 report_path = os.path.join(OUTPUT_DIR, 'evaluation_report.txt')
68 with open(report_path, 'w', encoding='utf-8') as f:

```

```

69         f.write(report_text)
70     print(f"\n{report_text}")
71     print(f"\n报告已保存: {report_path}")
72     print("步骤六完成! ")
73 if __name__ == '__main__':
74     main()

```

附录 H: 步骤七: V/C 热力图生成 (step7_heatmap.py)

```

1  import os
2  import math
3  import networkx as nx
4  import numpy as np
5  import pandas as pd
6  import matplotlib
7  matplotlib.use('Agg')
8  import matplotlib.pyplot as plt
9  import matplotlib.font_manager as fm
10 from matplotlib.lines import Line2D
11 fm._load_fontmanager(try_read_cache=False)
12 _cjk_fonts = [
13     'SimHei', 'Microsoft YaHei', 'SimSun', 'FangSong', 'KaiTi',
14     'STHeiti', 'STSong', 'STFangsong', 'STKaiti',
15     'Heiti TC', 'PingFang HK', 'PingFang SC',
16     'WenQuanYi Micro Hei', 'Noto Sans CJK SC', 'Droid Sans Fallback',
17 ]
18 _available = [f.name for f in fm.fontManager.ttflist]
19 _cjk = None
20 for font in _cjk_fonts:
21     if font in _available:
22         _cjk = font
23         break
24 if _cjk is None:
25     for font in _cjk_fonts:
26         for af in _available:
27             if font.lower() in af.lower() or af.lower() in font.lower():
28                 _cjk = af
29                 break
30     if _cjk:
31         break
32 if _cjk:
33     plt.rcParams['font.sans-serif'] = [_cjk] + plt.rcParams['font.sans-serif']
34     plt.rcParams['font.family'] = 'sans-serif'
35     print(f"[OK] 中文字体: {_cjk}")
36 else:
37     print("[WARN] 未找到中文字体")
38 plt.rcParams['axes.unicode_minus'] = False
39 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
40 STEP1_DIR = os.path.join(BASE_DIR, 'output_step1')

```



```

41 STEP3_DIR = os.path.join(BASE_DIR, 'output_step3')
42 STEP5_OPT_DIR = os.path.join(BASE_DIR, 'output_step5_minmax')
43 OUTPUT_DIR = os.path.join(BASE_DIR, 'output_step7')
44 os.makedirs(OUTPUT_DIR, exist_ok=True)
45 PERIODS = ['morning', 'noon', 'evening']
46 PERIOD_NAMES = {'morning': '早高峰', 'noon': '午高峰', 'evening': '晚高峰'}
47 BUILDING_LABEL = {
48     'T_16': '16幢', 'T_25': '25幢',
49     'T_1': 'T1', 'T_2': 'T2', 'T_3': 'T3', 'T_4': 'T4', 'T_5': 'T5',
50     'T_6': 'T6', 'T_7': 'T7', 'T_8': 'T8', 'T_9': 'T9', 'T_10': 'T10',
51     'T_1': 'T1', 'T_2': 'T2', 'T_3': 'T3', 'T_4': 'T4', 'T_5': 'T5',
52     'T_6': 'T6', 'T_7': 'T7', 'T_8': 'T8', 'T_9': 'T9', 'T_10': 'T10',
53     'TYGY': '桃园\n公寓', 'TYST': '桃园\n食堂',
54     'XYGY': '杏园\n公寓', 'XYST': '杏园\n食堂',
55     'GYGY': '桂苑\n公寓', 'GYST': '桂苑\n食堂', 'CYGY': '初阳\n公寓',
56     'SYJ': '商业街',
57 }
58 def load_vc_csv(path):
59     df = pd.read_csv(path)
60     vc = {}
61     cols = df.columns.tolist()
62     if 'from' in cols and 'to' in cols:
63         ucol, vcol = 'from', 'to'
64     else:
65         ucol, vcol = 'u', 'v'
66     if 'saturation' in cols:
67         vccol = 'saturation'
68     elif 'V/C' in cols:
69         vccol = 'V/C'
70     elif 'optimized_vc' in cols:
71         vccol = 'optimized_vc'
72     else:
73         return vc
74     for _, r in df.iterrows():
75         key = (str(r[ucol]), str(r[vcol]))
76         vc[key] = float(r[vccol])
77     return vc
78 def compute_layout(G):
79     geo_pos = {}
80     for n, data in G.nodes(data=True):
81         try:
82             lon = float(data.get('lon', ''))
83             lat = float(data.get('lat', ''))
84             if lon != 0 and lat != 0:
85                 geo_pos[n] = (lon, lat)
86         except (TypeError, ValueError):
87             pass
88     if len(geo_pos) >= max(5, int(0.6 * G.number_of_nodes())):
89         xs = np.array([p[0] for p in geo_pos.values()])
90         ys = np.array([p[1] for p in geo_pos.values()])

```

```

91     x_mid, y_mid = xs.mean(), ys.mean()
92     x_span = max(xs.max() - xs.min(), 1e-9)
93     y_span = max(ys.max() - ys.min(), 1e-9)
94     scale = 6.0 / max(x_span, y_span)
95     pos = {}
96     for n in G.nodes():
97         if n in geo_pos:
98             x, y = geo_pos[n]
99             pos[n] = ((x - x_mid) * scale, (y - y_mid) * scale)
100     missing = [n for n in G.nodes() if n not in pos]
101     if missing:
102         fallback = nx.spring_layout(G.to_undirected(), seed=42, k=2.0, iterations=300)
103         for n in missing:
104             pos[n] = (fallback[n][0] * 2.5, fallback[n][1] * 2.5)
105     return pos
106     return nx.kamada_kawai_layout(G.to_undirected())
107 def vc_color(vc):
108     if vc <= 0.35:
109         return (0.18, 0.72, 0.18, 0.9)
110     if vc <= 0.55:
111         return (0.50, 0.85, 0.18, 0.9)
112     if vc <= 0.75:
113         return (0.92, 0.78, 0.10, 0.9)
114     if vc <= 0.90:
115         return (0.95, 0.45, 0.08, 0.9)
116     if vc <= 1.00:
117         return (0.88, 0.12, 0.05, 0.95)
118     return (0.65, 0.0, 0.0, 1.0)
119 def draw_heatmap(ax, G, pos, vc_by_edge, title, is_left=True):
120     ax.set_title(title, fontsize=13, fontweight='bold', pad=8)
121     ax.axis('off')
122     xs = [p[0] for p in pos.values()]
123     ys = [p[1] for p in pos.values()]
124     pad_x = max((max(xs) - min(xs)) * 0.08, 0.2)
125     pad_y = max((max(ys) - min(ys)) * 0.08, 0.2)
126     ax.set_xlim(min(xs) - pad_x, max(xs) + pad_x)
127     ax.set_ylim(min(ys) - pad_y, max(ys) + pad_y)
128     ax.set_aspect('equal', adjustable='box')
129     for u, v, data in G.edges(data=True):
130         vc = vc_by_edge.get((u, v), 0.0)
131         ax.plot([pos[u][0], pos[v][0]], [pos[u][1], pos[v][1]],
132                 color=vc_color(vc), linewidth=0.8 + 5.0 * min(vc, 1.5), alpha=0.82, zorder=1)
133     for u, v, data in G.edges(data=True):
134         vc = vc_by_edge.get((u, v), 0.0)
135         width = float(data.get('width', 0))
136         if vc < 0.75 or width >= 99:
137             continue
138         x1, y1 = pos[u]
139         x2, y2 = pos[v]
140         mx, my = (x1 + x2) / 2, (y1 + y2) / 2

```

```

141     dx, dy = x2 - x1, y2 - y1
142     length = math.sqrt(dx * dx + dy * dy)
143     nx_dir, ny_dir = (-dy / length, dx / length) if length > 0 else (0, 0)
144     tx, ty = mx + nx_dir * 0.08, my + ny_dir * 0.08
145     color = '#900' if vc >= 1.0 else '#C0392B'
146     ax.annotate(f'{vc:.2f}', (tx, ty), fontsize=6.5, color=color, fontweight='bold',
147                ha='center', va='center', zorder=7,
148                bbox=dict(boxstyle='round,pad=0.2', facecolor='white',
149                          edgecolor=color, alpha=0.92, linewidth=0.8))
150 buildings = {n for n in G.nodes() if G.nodes[n].get('type', '') != '交叉口'}
151 for n in G.nodes():
152     if n in buildings:
153         x, y = pos[n]
154         nm = G.nodes[n].get('type', '')
155         c = {'教学楼': '#E74C3C', '宿舍': '#3498DB', '食堂': '#27AE60', '商业街': '#27AE60'}.
156             get(nm, '#888')
157         ax.scatter(x, y, s=160, c=c, edgecolors='white', linewidth=1.5, zorder=5)
158         label = BUILDING_LABEL.get(n, n)
159         ax.annotate(label, (x, y), textcoords='offset points', xytext=(0, -16),
160                    fontsize=7.5, ha='center', color='#222', fontweight='bold',
161                    bbox=dict(boxstyle='round,pad=0.3', facecolor='white', alpha=0.85), zorder
162                             =6)
163     else:
164         ax.scatter(pos[n][0], pos[n][1], s=8, c='CCCCCC', zorder=2)
165 legend_vc = [Line2D([0], [0], color=vc_color(x), lw=3, label=1)
166              for x, l in [(0.2, 'A 0.35'), (0.45, 'B'), (0.65, 'C'),
167                          (0.85, 'D'), (0.95, 'E'), (1.3, 'F >1.00')]]
168 if is_left:
169     ax.legend(handles=legend_vc, loc='lower left', fontsize=6.5, framealpha=0.85, title='V/C'
170             )
171 def summarize(period, base_vc, opt_vc, G):
172     real_edges = [(u, v) for u, v, d in G.edges(data=True) if float(d.get('width', 0)) < 99]
173     b_vals = [base_vc.get(e, 0.0) for e in real_edges]
174     o_vals = [opt_vc.get(e, 0.0) for e in real_edges]
175     b_max = max(b_vals) if b_vals else 0.0
176     o_max = max(o_vals) if o_vals else 0.0
177     b_high = sum(x > 0.75 for x in b_vals)
178     o_high = sum(x > 0.75 for x in o_vals)
179     return f"基准max={b_max:.2f}, 优化max={o_max:.2f}; 高负荷边 {b_high}+{o_high}"
180 def main():
181     G = nx.read_gml(os.path.join(STEP1_DIR, 'campus_graph.gml'))
182     pos = compute_layout(G)
183     for period in PERIODS:
184         print(f"{period} ...")
185         base_path = os.path.join(STEP3_DIR, f'link_flow_{period}.csv')
186         opt_path = os.path.join(STEP5_OPT_DIR, f'link_flow_optimized_{period}.csv')
187         if not os.path.exists(base_path):
188             raise FileNotFoundError(base_path)
189         if not os.path.exists(opt_path):
190             raise FileNotFoundError(opt_path)

```

```

188     base_vc = load_vc_csv(base_path)
189     opt_vc = load_vc_csv(opt_path)
190     summary = summarize(period, base_vc, opt_vc, G)
191     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(24, 11), facecolor='white')
192     draw_heatmap(ax1, G, pos, base_vc, f'UE基准 — {PERIOD_NAMES[period]}', True)
193     draw_heatmap(ax2, G, pos, opt_vc, f'MinMax V/C优化 — {PERIOD_NAMES[period]}', False)
194     plt.suptitle(f'校园路网 V/C 热力图 | {PERIOD_NAMES[period]} | {summary}', fontsize=15,
195                 fontweight='bold', y=0.99)
196     plt.tight_layout(rect=[0, 0.02, 1, 0.93])
197     out = os.path.join(OUTPUT_DIR, f'heatmap_{period}.png')
198     plt.savefig(out, dpi=220, bbox_inches='tight', facecolor='white')
199     plt.close()
200     print(f" {summary}")
201     print(f" -> {out}")
202     print(f"\n全部完成: {OUTPUT_DIR}/")
203 if __name__ == '__main__':
204     main()

```

附录 I: 步骤八: 功能区域划分 (step8_zones.py)

```

1  import networkx as nx, pandas as pd, os
2  STEP1_DIR = "output_step1"
3  OUTPUT_DIR = "output_step8"
4  os.makedirs(OUTPUT_DIR, exist_ok=True)
5  ZONES = {
6      'Z1': {'name': '16幢教学区', 'buildings': ['T_16'], 'A_eff': 450, 'type': '教学区'},
7      'Z2': {'name': '25幢教学区', 'buildings': ['T_25'], 'A_eff': 550, 'type': '教学区'},
8      'Z3': {'name': '桃园生活区', 'buildings': ['TYGY', 'TYST'], 'A_eff': 8160, 'type': '生活区'},
9      'Z4': {'name': '杏园生活区', 'buildings': ['XYGY', 'XYST'], 'A_eff': 5100, 'type': '生活区'},
10     'Z5': {'name': '桂苑生活区', 'buildings': ['GYGY', 'CYGY', 'GYST'], 'A_eff': 7700, 'type': '生活区'},
11     'Z6': {'name': '东区新教学楼', 'buildings': ['T_1', 'T_2', 'T_3', 'T_4', 'T_5'], 'A_eff': 2200, 'type':
12           '教学区'},
13     'Z7': {'name': '西区新教学楼', 'buildings': ['T_6', 'T_7', 'T_8', 'T_9', 'T_10'], 'A_eff': 4000, 'type':
14           '教学区'},
15     'Z8': {'name': '商业街', 'buildings': ['SYJ'], 'A_eff': 1200, 'type': '商业区'},
16 }
17 G_di = nx.read_gml(os.path.join(STEP1_DIR, "campus_graph.gml"))
18 G = G_di.to_undirected()
19 print(f"图: {G.number_of_nodes()} 节点, {G.number_of_edges()} 边(无向)")
20 def find_zone_roads(G, building_list, max_distance=2):
21     junctions = set(); edges_in_zone = []
22     for bld in building_list:
23         if bld not in G: continue
24         visited = {bld: 0}; queue = [bld]
25         while queue:
26             u = queue.pop(0); dist = visited[u]
27             if dist >= max_distance: continue
28             for v in G.neighbors(u):
29                 if v not in visited:

```

```

28         visited[v] = dist+1; queue.append(v)
29         ntype = str(G.nodes[v].get('type',''))
30         if ntype == '交叉口': junctions.add(v)
31     seen = set()
32     for u in junctions:
33         for v in G.neighbors(u):
34             if v in junctions:
35                 key = tuple(sorted([u,v]))
36                 if key in seen: continue
37                 seen.add(key)
38                 w = float(G[u][v].get('width',0))
39                 l = float(G[u][v].get('length',0))
40                 edges_in_zone.append({'from':u,'to':v,'width':w,'length':l})
41     return sorted(junctions), edges_in_zone
42 rows = []; building_to_zone = {}
43 for zid, zinfo in ZONES.items():
44     blids = zinfo['buildings']
45     junctions, edge_list = find_zone_roads(G, blids, max_distance=2)
46     for b in blids: building_to_zone[b] = zid
47     widths = [e['width'] for e in edge_list if e['width']>0 and e['width']<99]
48     min_w = min(widths) if widths else 0; max_w = max(widths) if widths else 0
49     avg_w = sum(widths)/len(widths) if widths else 0
50     rows.append({
51         'zone_id':zid,'zone_name':zinfo['name'],'zone_type':zinfo['type'],
52         'buildings':'|'.join(blids),'num_buildings':len(blids),'A_eff_m2':zinfo['A_eff'],
53         'junctions':'|'.join(junctions[:15]),'num_junctions':len(junctions),
54         'num_edges':len(edge_list),
55         'min_road_width':f"{min_w:.1f}",'max_road_width':f"{max_w:.1f}",'avg_road_width':f"{avg_w:.1f}",
56     })
57     print(f"{zid} {zinfo['name']}: {'|'.join(blids)} | "
58           f"路口={len(junctions)} 路段={len(edge_list)} 路宽={min_w:.0f}~{max_w:.0f}m A_eff={
59               zinfo['A_eff']}m²")
60 df = pd.DataFrame(rows)
61 df.to_csv(os.path.join(OUTPUT_DIR,"zones.csv"),index=False,encoding="utf-8-sig")
62 pd.DataFrame([{'building':b,'zone_id':z for b,z in building_to_zone.items()}]).to_csv(
63     os.path.join(OUTPUT_DIR,"building_zone_map.csv"),index=False,encoding="utf-8-sig")
64 print(f"\n共 {len(ZONES)} 个功能区 | 输出: {OUTPUT_DIR}/")

```

附录 J：步骤九：静态停车容量计算 (step9_static.py)

```

1 import networkx as nx, pandas as pd, numpy as np, os
2 STEP1_DIR="output_step1"; STEP8_DIR="output_step8"
3 OUTPUT_DIR="output_step9"; os.makedirs(OUTPUT_DIR,exist_ok=True)
4 a_e=1.08; beta=1.35
5 SAFETY_LAMBDA = {
6     'Z1': 0.72, 'Z2': 0.72,
7     'Z3': 0.63, 'Z4': 0.65, 'Z5': 0.63,
8     'Z6': 0.70, 'Z7': 0.70, 'Z8': 0.60,

```

```

9 }
10 G=nx.read_gml(os.path.join(STEP1_DIR,"campus_graph.gml"))
11 zones_df=pd.read_csv(os.path.join(STEP8_DIR,"zones.csv"))
12 print(f"{len(zones_df)} 区域")
13 print(f"{'区域':<12} {'A_raw':>7} ×{' ':>5} = {'A_safe':>7} → {'N_static':>7}")
14 print("-" * 52)
15 rows=[]
16 for _,zrow in zones_df.iterrows():
17     zid=zrow['zone_id'];zname=zrow['zone_name']
18     A_safe=float(zrow['A_eff_m2'])
19     lam = SAFETY_LAMBDA.get(zid, 0.70)
20     A_raw = A_safe / lam
21     N_static=int(np.floor(A_safe/(beta*a_e)))
22     rows.append({
23         'zone_id':zid,'zone_name':zname,
24         'A_raw':round(A_raw,0),'lambda':lam,'A_eff_m2':A_safe,
25         'N_static':N_static,'binding_constraint':'面积',
26     })
27     print(f"{'zname':<12} {'A_raw':>7.0f} ×{'lambda':<5.2f} = {'A_safe':>7.0f} → {'N_static':>7}")
28 df=pd.DataFrame(rows)
29 df.to_csv(os.path.join(OUTPUT_DIR,"static_capacity.csv"),index=False,encoding="utf-8-sig")
30 total=sum(r['N_static'] for r in rows)
31 print(f"\nΣ N_static = {total} 辆 | 输出: {OUTPUT_DIR}/")

```

附录 K：步骤十：动态道路容量计算 (step10_dynamic.py)

```

1 import pandas as pd, numpy as np, os
2 STEP3_DIR="output_step3"; STEP8_DIR="output_step8"
3 OUTPUT_DIR="output_step10"; os.makedirs(OUTPUT_DIR,exist_ok=True)
4 SAT_LIMIT=0.85; PCU_BIKE=0.30; PERIODS=['morning','noon','evening']
5 ALPHA_DEFAULT={'morning':0.90,'noon':0.80,'evening':0.90}
6 zones_df=pd.read_csv(os.path.join(STEP8_DIR,"zones.csv"))
7 link_flows={}
8 for p in PERIODS:
9     fp=os.path.join(STEP3_DIR,f"link_flow_{p}.csv")
10    if os.path.exists(fp):link_flows[p]=pd.read_csv(fp)
11    print(f"加载 {p}: {len(link_flows.get(p,[]))} 条")
12 rows=[]
13 for _,zrow in zones_df.iterrows():
14     zid=zrow['zone_id'];zname=zrow['zone_name']
15     junctions=set(j.strip() for j in str(zrow['junctions']).split('|') if j.strip())
16     period_results={}
17     for period in PERIODS:
18         lf=link_flows.get(period)
19         if lf is None:continue
20         mask=lf['from'].isin(junctions)|lf['to'].isin(junctions)
21         zone_edges=lf[mask]
22         if len(zone_edges)==0:
23             period_results[period]={'C_road':0,'N_dyn':0}

```

```

24         continue
25     remaining_bike=[]
26     for _,edge in zone_edges.iterrows():
27         cap=float(edge['capacity']);flow_eq=float(edge['flow_eq'])
28         remaining=max(0.0,cap*SAT_LIMIT-flow_eq)/PCU_BIKE
29         remaining_bike.append({'remaining_ebike':remaining,'edge':f"{edge['from']}->{edge['to']}"})
30     rv=sorted([r['remaining_ebike'] for r in remaining_bike])
31     bi=max(0,min(len(rv)-1,int(len(rv)*0.05)))
32     C_road=rv[bi] if rv else 0
33     bneck=min(remaining_bike,key=lambda r:r['remaining_ebike']) if remaining_bike else None
34     alpha=ALPHA_DEFAULT[period]
35     N_dyn=C_road/alpha if alpha>0 else float('inf')
36     period_results[period]={ 'C_road':C_road, 'N_dyn':N_dyn, 'num_edges':len(zone_edges), '
        bottleneck':bneck['edge'] if bneck else '-'}
37     dyn_vals=[v['N_dyn'] for v in period_results.values() if v['N_dyn']>0]
38     N_dynamic=int(np.floor(min(dyn_vals))) if dyn_vals else 0
39     binding=min(period_results,key=lambda p:period_results[p]['N_dyn']) if period_results else '-'
40     row={'zone_id':zid,'zone_name':zname,'N_dynamic':N_dynamic,'binding_period':binding}
41     for period in PERIODS:
42         pr=period_results.get(period,{})
43         row[f'{period}_C_road']=f"{pr.get('C_road',0):.0f}"
44         row[f'{period}_N_dyn']=f"{pr.get('N_dyn',0):.0f}"
45         row[f'{period}_bottleneck']=pr.get('bottleneck','-')
46     rows.append(row)
47     print(f"{zid} {zname}: N_dynamic={N_dynamic} (卡在{binding}) "
48           f"【早={period_results.get('morning',{}).get('N_dyn',0):.0f} "
49           f"午={period_results.get('noon',{}).get('N_dyn',0):.0f} "
50           f"晚={period_results.get('evening',{}).get('N_dyn',0):.0f}】")
51 df=pd.DataFrame(rows)
52 df.to_csv(os.path.join(OUTPUT_DIR,"dynamic_capacity.csv"),index=False,encoding="utf-8-sig")
53 print(f"\nΣ N_dynamic = {sum(r['N_dynamic'] for r in rows)} | 输出: {OUTPUT_DIR}/")

```

附录 L：步骤十一：综合承载力求解 (step11_solve.py)

```

1  import pandas as pd, numpy as np, os
2  STEP2_DIR="output_step2"; STEP8_DIR="output_step8"
3  STEP9_DIR="output_step9"; STEP10_DIR="output_step10"
4  OUTPUT_DIR="output_step11"; os.makedirs(OUTPUT_DIR,exist_ok=True)
5  PERIODS=['morning','noon','evening']
6  static_df=pd.read_csv(os.path.join(STEP9_DIR,"static_capacity.csv"))
7  dyn_df=pd.read_csv(os.path.join(STEP10_DIR,"dynamic_capacity.csv"))
8  zone_map=pd.read_csv(os.path.join(STEP8_DIR,"building_zone_map.csv"))
9  b2z=dict(zip(zone_map['building'],zone_map['zone_id']))
10 od_bike={}
11 for p in PERIODS:
12     fp=os.path.join(STEP2_DIR,f"od_{p}_b.csv")
13     if os.path.exists(fp):od_bike[p]=pd.read_csv(fp,index_col=0)

```

```

14 zone_demand={}
15 for p in PERIODS:
16     ob=od_bike.get(p)
17     if ob is None:continue
18     for bld in ob.columns:
19         zid=b2z.get(bld,''); arr=ob.loc[:,bld].sum() if bld in ob.columns else 0
20         if zid:zone_demand[zid]=max(zone_demand.get(zid,0),arr)
21 rows=[]
22 for _,sr in static_df.iterrows():
23     zid=sr['zone_id'];zname=sr['zone_name'];N_static=int(sr['N_static'])
24     dr=dyn_df[dyn_df['zone_id']==zid]
25     N_dynamic=int(dr.iloc[0]['N_dynamic']) if len(dr)>0 else N_static
26     binding=dr.iloc[0]['binding_period'] if len(dr)>0 else '-'
27     N_opt=min(N_static,N_dynamic)
28     limiting='静态面积' if N_static<=N_dynamic else f'动态({binding})'
29     D_i=zone_demand.get(zid,0);rho=D_i/N_opt if N_opt>0 else float('inf')
30     if rho<0.85:status='容量充足'
31     elif rho<=1.00:status='接近饱和'
32     else:status='超载'
33     rows.append({'zone_id':zid,'zone_name':zname,'N_static':N_static,'N_dynamic':N_dynamic,
34                 'N_opt':N_opt,'limiting':limiting,'D_i':f"{D_i:.0f}",'rho':f"{rho:.3f}",'status':
35                     status})
36     m=' ' if N_static!=N_dynamic else ''
37     print(f"{zid} {zname:8s}: N_static={N_static:4d} N_dynamic={N_dynamic:4d} N*={N_opt:4d} D={
38         D_i:5.0f} = {rho:.3f} [{status}]{m}")
39 N_max=sum(r['N_opt'] for r in rows)
40 period_demand={p:0.0 for p in PERIODS}
41 for p in PERIODS:
42     ob=od_bike.get(p)
43     if ob is None:continue
44     for bld in ob.columns:
45         if bld in b2z:period_demand[p]+=ob.loc[:,bld].sum()
46 worst_p=max(period_demand,key=period_demand.get)
47 theta=period_demand[worst_p]/N_max if N_max>0 else 0
48 print(f"\nN_max = {N_max} 辆 | = {theta:.3f} (最差:{worst_p})")
49 bottleneck=[r for r in rows if r['N_dynamic']<r['N_static']]
50 if bottleneck:
51     for r in bottleneck:
52         pct=(1-r['N_opt']/max(r['N_static'],1))*100
53         print(f"瓶颈: {r['zone_name']} {r['N_static']}→{r['N_opt']} (-{pct:.0f}%)")
54 df=pd.DataFrame(rows)
55 df.to_csv(os.path.join(OUTPUT_DIR,"final_capacity.csv"),index=False,encoding="utf-8-sig")
56 with open(os.path.join(OUTPUT_DIR,"capacity_report.txt"),"w",encoding="utf-8") as f:
57     f.write(f"校园电动车环境承载力评估 (v2)\n")
58     f.write(f"N_max = {N_max} 辆 | = {theta:.3f}\n\n")
59     for r in rows:
60         f.write(f"{r['zone_name']}: N*={r['N_opt']} = {r['rho']} [{r['status']}] \n")
61 print(f"输出: {OUTPUT_DIR}/")

```


附录 M：步骤十二：问题三数据准备 (step12_prepare.py)

```
1 import networkx as nx
2 import pandas as pd
3 import numpy as np
4 import os
5 STEP1_DIR = "output_step1"
6 STEP2_DIR = "output_step2"
7 STEP3_DIR = "output_step3"
8 STEP8_DIR = "output_step8"
9 STEP11_DIR = "output_step11"
10 OUTPUT_DIR = "output_step12"
11 os.makedirs(OUTPUT_DIR, exist_ok=True)
12 PERIODS = ['morning', 'noon', 'evening']
13 PERIOD_LABEL = {'morning': '早高峰', 'noon': '午高峰', 'evening': '晚高峰'}
14 print("1. 加载路网...")
15 G = nx.read_gml(os.path.join(STEP1_DIR, "campus_graph.gml"))
16 print(f" 节点: {G.number_of_nodes()}, 边: {G.number_of_edges()}")
17 print("2. 加载区域数据...")
18 zone_map = pd.read_csv(os.path.join(STEP8_DIR, "building_zone_map.csv"))
19 zones_df = pd.read_csv(os.path.join(STEP8_DIR, "zones.csv"))
20 b2z = dict(zip(zone_map['building'], zone_map['zone_id']))
21 z2name = dict(zip(zones_df['zone_id'], zones_df['zone_name']))
22 z2type = dict(zip(zones_df['zone_id'], zones_df['zone_type']))
23 building_nodes = [n for n in G.nodes() if str(G.nodes[n].get('type', '')) != '交叉口']
24 print(f" 建筑节点: {len(building_nodes)}")
25 print("3. 加载 OD 数据...")
26 od_total = {}
27 od_ebike = {}
28 for period in PERIODS:
29     fp_t = os.path.join(STEP2_DIR, f"od_{period}_total.csv")
30     fp_b = os.path.join(STEP2_DIR, f"od_{period}_b.csv")
31     if os.path.exists(fp_t):
32         od_total[period] = pd.read_csv(fp_t, index_col=0)
33     if os.path.exists(fp_b):
34         od_ebike[period] = pd.read_csv(fp_b, index_col=0)
35     print(f" {period}: total={od_total[period].shape} if period in od_total else 'N/A', "
36           f"ebike={od_ebike[period].shape} if period in od_ebike else 'N/A'")
37 print("4. 加载路段流量数据...")
38 link_flows = {}
39 for period in PERIODS:
40     fp = os.path.join(STEP3_DIR, f"link_flow_{period}.csv")
41     if os.path.exists(fp):
42         link_flows[period] = pd.read_csv(fp)
43         print(f" {period}: {len(link_flows[period])} 条路段")
44 print("5. 加载问题二结果...")
45 capacity_df = pd.read_csv(os.path.join(STEP11_DIR, "final_capacity.csv"))
46 print("\n6. 构建节点级基础数据表...")
47 rows = []
48 for node in G.nodes():
```

```

49 ntype = str(G.nodes[node].get('type', ''))
50 zid = b2z.get(node, '')
51 zname = z2name.get(zid, '')
52 ztype = z2type.get(zid, '')
53 row = {
54     'node_id': node,
55     'node_type': ntype,
56     'zone_id': zid,
57     'zone_name': zname,
58     'zone_type': ztype,
59 }
60 for period in PERIODS:
61     ot = od_total.get(period)
62     ob = od_ebike.get(period)
63     if ot is not None and node in ot.index:
64         row[f'demand_{period}_total'] = ot.loc[node].sum()
65     else:
66         row[f'demand_{period}_total'] = 0
67     if ob is not None and node in ob.index:
68         row[f'demand_{period}_ebike'] = ob.loc[node].sum()
69     else:
70         row[f'demand_{period}_ebike'] = 0
71 for period in PERIODS:
72     lf = link_flows.get(period)
73     if lf is not None:
74         mask = (lf['from'] == node) | (lf['to'] == node)
75         nearby = lf[mask]
76         if len(nearby) > 0:
77             row[f'vc_max_{period}'] = nearby['saturation'].astype(float).max()
78             row[f'risk_max_{period}'] = nearby['risk'].astype(float).max()
79         else:
80             row[f'vc_max_{period}'] = 0
81             row[f'risk_max_{period}'] = 0
82     else:
83         row[f'vc_max_{period}'] = 0
84         row[f'risk_max_{period}'] = 0
85 if node in building_nodes:
86     cap_row = capacity_df[capacity_df['zone_id'] == zid]
87     if len(cap_row) > 0:
88         row['N_opt'] = int(cap_row.iloc[0]['N_opt'])
89         row['rho'] = float(cap_row.iloc[0]['rho'])
90         row['status'] = cap_row.iloc[0]['status']
91     else:
92         row['N_opt'] = 0
93         row['rho'] = 0
94         row['status'] = ''
95 else:
96     row['N_opt'] = 0
97     row['rho'] = 0
98     row['status'] = ''

```

```

99     rows.append(row)
100 df_nodes = pd.DataFrame(rows)
101 df_nodes.to_csv(os.path.join(OUTPUT_DIR, "base_data_summary.csv"),
102                 index=False, encoding="utf-8-sig")
103 building_rows = df_nodes[df_nodes['node_type'] != '交叉口']
104 print(f"\n 建筑节点: {len(building_rows)} 个")
105 print(f" 各区域需求与压力:")
106 for _, r in building_rows.iterrows():
107     dem = sum(float(r[f'demand_{p}_total']) for p in PERIODS)
108     ebk = sum(float(r[f'demand_{p}_ebike']) for p in PERIODS)
109     print(f"    {r['node_id']:8s} [{r['zone_name']:6s}] "
110           f"总需求={dem:6.0f} ebike={ebk:5.0f} "
111           f"V/C早={r['vc_max_morning']:.3f} "
112           f"={r['rho']:.3f}")
113 print("\n7. 构建区域级 e-bike OD 聚合表...")
114 zone_list = sorted(zones_df['zone_id'].tolist())
115 od_zone_rows = []
116 for period in PERIODS:
117     ob = od_ebike.get(period)
118     if ob is None:
119         continue
120     for zi in zone_list:
121         bi = [b for b, z in b2z.items() if z == zi and b in ob.index]
122         for zj in zone_list:
123             bj = [b for b, z in b2z.items() if z == zj and b in ob.columns]
124             flow_sum = 0
125             for bi_node in bi:
126                 for bj_node in bj:
127                     if bi_node in ob.index and bj_node in ob.columns:
128                         flow_sum += ob.loc[bi_node, bj_node]
129             if flow_sum > 0.1:
130                 od_zone_rows.append({
131                     'period': period,
132                     'from_zone': zi,
133                     'to_zone': zj,
134                     'from_name': z2name.get(zi, ''),
135                     'to_name': z2name.get(zj, ''),
136                     'ebike_flow': flow_sum,
137                 })
138 df_od = pd.DataFrame(od_zone_rows)
139 df_od.to_csv(os.path.join(OUTPUT_DIR, "od_ebike_summary.csv"),
140             index=False, encoding="utf-8-sig")
141 for period in PERIODS:
142     pdf = df_od[df_od['period'] == period].sort_values('ebike_flow', ascending=False)
143     if len(pdf) == 0:
144         continue
145     print(f"\n {PERIOD_LABEL[period]} e-bike 主要流向 (Top 5):")
146     for _, r in pdf.head(5).iterrows():
147         print(f"    {r['from_name']:8s} → {r['to_name']:8s} {r['ebike_flow']:6.0f}")
148 print(f"\n{' '*60}")

```

```

149 print(f"完成! 输出:")
150 print(f" {OUTPUT_DIR}/base_data_summary.csv ({len(df_nodes)} 行, 节点级)")
151 print(f" {OUTPUT_DIR}/od_ebike_summary.csv ({len(df_od)} 行, 区域级)")
152 print(f"\n关键数据供后续步骤使用:")
153 print(f" 建筑节点数: {len(building_rows)}")
154 print(f" 区域数: {len(zone_list)}")
155 print(f" 时段数: {len(PERIODS)}")

```

附录 N: 步骤十三: 候选站点构造与评价 (step13_candidate_stops.py)

```

1 import networkx as nx
2 import pandas as pd
3 import numpy as np
4 import os
5 STEP1_DIR = "output_step1"
6 STEP3_DIR = "output_step3"
7 STEP8_DIR = "output_step8"
8 STEP12_DIR = "output_step12"
9 OUTPUT_DIR = "output_step13"
10 os.makedirs(OUTPUT_DIR, exist_ok=True)
11 PERIODS = ['morning', 'noon', 'evening']
12 G = nx.read_gml(os.path.join(STEP1_DIR, "campus_graph.gml"))
13 zones_df = pd.read_csv(os.path.join(STEP8_DIR, "zones.csv"))
14 base_df = pd.read_csv(os.path.join(STEP12_DIR, "base_data_summary.csv"))
15 link_flows = {}
16 for period in PERIODS:
17     fp = os.path.join(STEP3_DIR, f"link_flow_{period}.csv")
18     if os.path.exists(fp):
19         link_flows[period] = pd.read_csv(fp)
20 print(f"路网: {G.number_of_nodes()} 节点")
21 print(f"区域: {len(zones_df)} 个")
22 CANDIDATE_STOPS = [
23     {
24         'stop_id': 'S1', 'stop_name': '桂苑站',
25         'nearest_node': 'N28', 'zone_id': 'Z5',
26         'reason': '桂苑生活区核心, GYST+GYGY+CYGY复合需求',
27     },
28     {
29         'stop_id': 'S2', 'stop_name': '桃园站',
30         'nearest_node': 'N4', 'zone_id': 'Z3',
31         'reason': '桃园生活区核心, TYGY+TYST+近T_3/T_4',
32     },
33     {
34         'stop_id': 'S3', 'stop_name': '杏园站',
35         'nearest_node': 'N12', 'zone_id': 'Z4',
36         'reason': '杏园生活区核心, XYGY+XYST+商业街方向',
37     },
38     {
39         'stop_id': 'S4', 'stop_name': '16幢站',

```

```

40     'nearest_node': 'N15', 'zone_id': 'Z1',
41     'reason': '16幢教学区核心, 近T_2/T_5',
42 },
43 {
44     'stop_id': 'S5', 'stop_name': '25幢站',
45     'nearest_node': 'N48', 'zone_id': 'Z2',
46     'reason': '25幢教学区核心, 近T_9/T_10',
47 },
48 {
49     'stop_id': 'S6', 'stop_name': '商业街站',
50     'nearest_node': 'N13', 'zone_id': 'Z8',
51     'reason': 'SYJ商业街, 早晚高峰最大目的地',
52 },
53 {
54     'stop_id': 'S7', 'stop_name': '东区教学楼站',
55     'nearest_node': 'N20', 'zone_id': 'Z6',
56     'reason': '东区新教学楼核心(T_1~T_3), 早高峰主要目的地',
57 },
58 {
59     'stop_id': 'S8', 'stop_name': '西区教学楼站',
60     'nearest_node': 'N42', 'zone_id': 'Z7',
61     'reason': '西区新教学楼核心(T_6~T_8), 早高峰主要目的地',
62 },
63 ]
64 def get_edge_conditions(link_df, node):
65     mask = (link_df['from'] == node) | (link_df['to'] == node)
66     nearby = link_df[mask]
67     if len(nearby) == 0:
68         return {'vc': 0, 'risk': 0, 'width': 8}
69     return {
70         'vc': float(nearby['saturation'].max()),
71         'risk': float(nearby['risk'].max()),
72         'width': float(nearby['width_m'].max()) if 'width_m' in nearby.columns else 8.0,
73     }
74 rows = []
75 for stop in CANDIDATE_STOPS:
76     node = stop['nearest_node']
77     zone = zones_df[zones_df['zone_id'] == stop['zone_id']]
78     zname = zone.iloc[0]['zone_name'] if len(zone) > 0 else ''
79     cond_m = get_edge_conditions(link_flows.get('morning', pd.DataFrame()), node)
80     cond_n = get_edge_conditions(link_flows.get('noon', pd.DataFrame()), node)
81     cond_e = get_edge_conditions(link_flows.get('evening', pd.DataFrame()), node)
82     vc_max = max(cond_m['vc'], cond_n['vc'], cond_e['vc'])
83     risk_max = max(cond_m['risk'], cond_n['risk'], cond_e['risk'])
84     can_stop = (cond_m['width'] >= 5.0) and (vc_max <= 0.90)
85     if not can_stop:
86         fail_reason = []
87         if cond_m['width'] < 5.0:
88             fail_reason.append(f"路宽不足({cond_m['width']:.1f}m)")
89         if vc_max > 0.90:

```

```

90         fail_reason.append(f"拥堵(V/C={vc_max:.3f})")
91     reason_str = '; '.join(fail_reason)
92     else:
93         reason_str = '符合条件'
94     rows.append({
95         'stop_id': stop['stop_id'],
96         'stop_name': stop['stop_name'],
97         'nearest_node': node,
98         'zone_id': stop['zone_id'],
99         'zone_name': zname,
100        'road_width_m': cond_m['width'],
101        'vc_morning': f"{cond_m['vc']:.3f}",
102        'vc_noon': f"{cond_n['vc']:.3f}",
103        'vc_evening': f"{cond_e['vc']:.3f}",
104        'risk_morning': f"{cond_m['risk']:.0f}",
105        'risk_noon': f"{cond_n['risk']:.0f}",
106        'risk_evening': f"{cond_e['risk']:.0f}",
107        'vc_max': f"{vc_max:.3f}",
108        'can_stop': can_stop,
109        'stop_reason': reason_str if not can_stop else stop['reason'],
110    })
111    print(f"{stop['stop_id']} {stop['stop_name']:10s} @ {node:6s} "
112          f"路宽={cond_m['width']:.0f}m V/C={vc_max:.3f} "
113          f"{' ' if can_stop else ' ' + reason_str}")
114    df = pd.DataFrame(rows)
115    df.to_csv(os.path.join(OUTPUT_DIR, "candidate_stops.csv"), index=False, encoding="utf-8-sig")
116    valid = [r for r in rows if r['can_stop']]
117    print(f"\n{'='*60}")
118    print(f"候选站点: {len(rows)} 个, 符合条件: {len(valid)} 个")
119    print(f"输出: {OUTPUT_DIR}/candidate_stops.csv")

```

附录 O：步骤十四：MCLP 最大覆盖选址 (step14_coverage.py)

```

1  import networkx as nx
2  import pandas as pd
3  import numpy as np
4  import os
5  from itertools import combinations
6  STEP1_DIR = "output_step1"
7  STEP3_DIR = "output_step3"
8  STEP12_DIR = "output_step12"
9  STEP13_DIR = "output_step13"
10 OUTPUT_DIR = "output_step14"
11 os.makedirs(OUTPUT_DIR, exist_ok=True)
12 R0 = 400
13 PHI_MIN = 0.35
14 W_WIDTH = 0.3
15 W_VC = 0.4
16 W_RISK = 0.3

```

```

17 B_MAX = 8
18 P_MIN = 0.55
19 MIN_STOPS = 5
20 G = nx.read_gml(os.path.join(STEP1_DIR, "campus_graph.gml"))
21 stops_df = pd.read_csv(os.path.join(STEP13_DIR, "candidate_stops.csv"))
22 base_df = pd.read_csv(os.path.join(STEP12_DIR, "base_data_summary.csv"))
23 lf_morning = pd.read_csv(os.path.join(STEP3_DIR, "link_flow_morning.csv"))
24 building_df = base_df[base_df['node_type'] != '交叉口'].copy()
25 building_nodes = building_df['node_id'].tolist()
26 stop_nodes = [stops_df.iloc[i] for i in range(len(stops_df))]
27 print(f"需求点(建筑): {len(building_nodes)}")
28 print(f"候选站点: {len(stop_nodes)}")
29 print(f"覆盖半径 R0 = {R0}m, 最低适宜性 _min = {PHI_MIN}\n")
30 print("2. 计算覆盖矩阵...")
31 coverage_matrix = {}
32 coverage_records = []
33 for _, bld_row in building_df.iterrows():
34     bi = bld_row['node_id']
35     if bi not in G:
36         continue
37     for _, st_row in stops_df.iterrows():
38         sj_node = st_row['nearest_node']
39         if sj_node not in G:
40             continue
41         try:
42             dist = nx.shortest_path_length(G, bi, sj_node, weight='length')
43         except nx.NetworkXNoPath:
44             dist = float('inf')
45         covered = 1 if dist <= R0 else 0
46         coverage_matrix[(bi, st_row['stop_id'])] = dist
47         coverage_records.append({
48             'building': bi,
49             'stop_id': st_row['stop_id'],
50             'stop_name': st_row['stop_name'],
51             'distance_m': f"{dist:.0f}" if dist < float('inf') else '∞',
52             'covered': covered,
53         })
54 df_cov = pd.DataFrame(coverage_records)
55 df_cov.to_csv(os.path.join(OUTPUT_DIR, "stop_coverage.csv"), index=False, encoding="utf-8-sig")
56 print(" 覆盖矩阵 (建筑 → 站点, 距离):")
57 print(f" {'建筑':<8}", end="")
58 for s in stops_df['stop_id']:
59     print(f" {s:>6}", end="")
60 print(f" {'覆盖范围':>6}")
61 for bi in building_nodes:
62     print(f" {bi:<8}", end="")
63     for _, st in stops_df.iterrows():
64         d = coverage_matrix.get((bi, st['stop_id']), float('inf'))
65         mark = ' ' if d <= R0 else ' '
66         print(f" {d:3.0f}{mark} ", end="")

```

```

67     nc = sum(1 for _, st in stops_df.iterrows()
68             if coverage_matrix.get((bi, st['stop_id']), float('inf')) <= R0)
69     print(f" {nc}/{len(stop_nodes)}")
70 print("\n3. 计算站点适宜性...")
71 V_MAX = lf_morning['saturation'].astype(float).max()
72 S_MAX = lf_morning['risk'].astype(float).max()
73 W_MAX = 8.0
74 suit_rows = []
75 for _, st in stops_df.iterrows():
76     node = st['nearest_node']
77     mask = (lf_morning['from'] == node) | (lf_morning['to'] == node)
78     nearby = lf_morning[mask]
79     if len(nearby) > 0:
80         w_j = float(nearby['width_m'].max()) if 'width_m' in nearby.columns else 8.0
81         v_j = float(nearby['saturation'].max())
82         s_j = float(nearby['risk'].max())
83     else:
84         w_j, v_j, s_j = 8.0, 0.0, 0.0
85     phi = (W_WIDTH * (w_j / max(W_MAX, 1e-9))
86           + W_VC * (1 - v_j / max(V_MAX, 1e-9))
87           + W_RISK * (1 - s_j / max(S_MAX, 1e-9)))
88     suitable = 1 if phi >= PHI_MIN else 0
89     suit_rows.append({
90         'stop_id': st['stop_id'],
91         'stop_name': st['stop_name'],
92         'road_width': w_j,
93         'vc_morning': v_j,
94         'risk_morning': s_j,
95         'phi_width': W_WIDTH * w_j / max(W_MAX, 1e-9),
96         'phi_vc': W_VC * (1 - v_j / max(V_MAX, 1e-9)),
97         'phi_risk': W_RISK * (1 - s_j / max(S_MAX, 1e-9)),
98         'phi': f"{phi:.4f}",
99         'suitable': suitable,
100     })
101     print(f" {st['stop_id']} {st['stop_name']:10s}: ={phi:.4f} "
102           f"(路宽={w_j:.0f}m V/C={v_j:.3f} 风险={s_j:.0f}) "
103           f"{' ' if suitable else ' 不适宜'}")
104 df_suit = pd.DataFrame(suit_rows)
105 df_suit.to_csv(os.path.join(OUTPUT_DIR, "stop_suitability.csv"), index=False, encoding="utf-8-
106               sig")
107 print("\n4. MCLP 最大覆盖选址...")
108 BUILDING_CAPACITY = {
109     'T_16': 3000, 'T_25': 2000,
110     'T_1': 1000, 'T_2': 1000, 'T_3': 1000, 'T_4': 1000, 'T_5': 1000,
111     'T_6': 1000, 'T_7': 1000, 'T_8': 1000, 'T_9': 1000, 'T_10': 1000,
112     'TYGY': 13000, 'XYGY': 7000, 'CYGY': 3000, 'GYGY': 5000,
113     'TYST': 1100, 'XYST': 1100, 'GYST': 1100, 'SYJ': 5000,
114 }
115 demands = {}
116 for _, bld in building_df.iterrows():

```



```

116     bi = bld['node_id']
117     demands[bi] = BUILDING_CAPACITY.get(bi, 500)
118     total_demand = sum(demands.values())
119     print(f" 总接驳需求(早高峰): {total_demand:.0f} 人次")
120     stop_ids = [st['stop_id'] for _, st in stops_df.iterrows()]
121     a = {}
122     for _, st in stops_df.iterrows():
123         sj_id = st['stop_id']
124         suit_row = [r for r in suit_rows if r['stop_id'] == sj_id]
125         suitable = suit_row[0]['suitable'] if suit_row else 0
126         for bi in building_nodes:
127             d = coverage_matrix.get((bi, sj_id), float('inf'))
128             a[(bi, sj_id)] = 1 if (d <= R0 and suitable) else 0
129     best_cov_rate = 0
130     best_k = 0
131     best_selection = []
132     for k in range(MIN_STOPS, min(B_MAX + 1, len(stop_ids) + 1)):
133         for combo in combinations(stop_ids, k):
134             covered_demand = 0
135             for bi in building_nodes:
136                 if any(a.get((bi, sj), 0) for sj in combo):
137                     covered_demand += demands.get(bi, 0)
138             cov_rate = covered_demand / max(total_demand, 1)
139             if cov_rate < P_MIN:
140                 continue
141             better = (cov_rate > best_cov_rate + 1e-6 or
142                     (abs(cov_rate - best_cov_rate) < 1e-6 and k < best_k))
143             if better or not best_selection:
144                 best_cov_rate = cov_rate
145                 best_k = k
146                 best_selection = list(combo)
147     print(f" 枚举组合数: {sum(1 for k in range(MIN_STOPS, min(B_MAX+1, len(stop_ids)+1)) for _ in
148         combinations(stop_ids, k))}")
149     print(f" 最优: 覆盖率={best_cov_rate:.1%}, 用 {best_k} 站: {best_selection}")
150     sel_rows = []
151     for _, st in stops_df.iterrows():
152         selected = st['stop_id'] in best_selection
153         cov_dem = sum(demands.get(bi, 0) for bi in building_nodes
154             if a.get((bi, st['stop_id']), 0))
155         phi_val = float([r['phi'] for r in suit_rows if r['stop_id'] == st['stop_id']][0])
156         sel_rows.append({
157             'stop_id': st['stop_id'],
158             'stop_name': st['stop_name'],
159             'nearest_node': st['nearest_node'],
160             'zone_name': st['zone_name'],
161             'vc_max': st['vc_max'],
162             'phi': f"{phi_val:.4f}",
163             'covered_demand': f"{cov_dem:.0f}",
164             'covered_pct': f"{cov_dem/max(total_demand,1)*100:.1f}%",
165             'selected': selected,

```

```

165     })
166     print(f" {st['stop_id']} {st['stop_name']:10s}: ={phi_val:.4f}, "
167           f"覆盖需求={cov_dem:.0f}({cov_dem/max(total_demand,1)*100:.1f}%), "
168           f"{' ' if selected else ' '}")
169 df_sel = pd.DataFrame(sel_rows)
170 df_sel.to_csv(os.path.join(OUTPUT_DIR, "selected_stops.csv"), index=False, encoding="utf-8-sig"
171               )
172 print(f"\n{'='*60}")
173 print(f"MCLP 结果: 从 {len(stop_ids)} 个候选站中选出 {len(best_selection)} 个")
174 print(f"最优覆盖: {best_cov_rate:.1}%")
175 print(f"选中: {' '.join(best_selection)}")
176 print(f"输出: {OUTPUT_DIR}/")

```

附录 P: 步骤十五: BPR 阻抗线路规划 (step15_routes.py)

```

1 import networkx as nx
2 import pandas as pd
3 import numpy as np
4 import os
5 import math
6 from collections import defaultdict
7 STEP1_DIR = "output_step1"
8 STEP3_DIR = "output_step3"
9 STEP13_DIR = "output_step13"
10 STEP14_DIR = "output_step14"
11 OUTPUT_DIR = "output_step15"
12 os.makedirs(OUTPUT_DIR, exist_ok=True)
13 DELTA = 0.5
14 BUS_SPEED = 20
15 ALPHA_L = 0.2
16 ALPHA_T = 0.3
17 ALPHA_V = 0.2
18 ALPHA_S = 0.2
19 ALPHA_P = 0.1
20 G = nx.read_gml(os.path.join(STEP1_DIR, "campus_graph.gml"))
21 stops_df = pd.read_csv(os.path.join(STEP13_DIR, "candidate_stops.csv"))
22 sel_df = pd.read_csv(os.path.join(STEP14_DIR, "selected_stops.csv"))
23 lf = pd.read_csv(os.path.join(STEP3_DIR, "link_flow_morning.csv"))
24 selected = sel_df[sel_df['selected'] == True]
25 print(f"选中站点: {len(selected)} 个")
26 for _, s in selected.iterrows():
27     print(f" {s['stop_id']} {s['stop_name']} @ {s['nearest_node']}")
28 stop_to_node = dict(zip(stops_df['stop_id'], stops_df['nearest_node']))
29 stop_name = dict(zip(stops_df['stop_id'], stops_df['stop_name']))
30 CANDIDATE_ROUTES = [
31     {
32         'route_id': 'R1',
33         'route_name': '教学潮汐线',
34         'route_type': '潮汐线 桃园→16幢→25幢→杏园→桂苑',

```

```

35     'stop_sequence': ['S2', 'S4', 'S5', 'S3', 'S1'],
36     'direction_morning': 'forward',
37     'direction_noon': 'reverse',
38 },
39 {
40     'route_id': 'R2',
41     'route_name': '校园环线',
42     'route_type': '环线 桂苑→桃园→16幢→25幢→杏园→桂苑',
43     'stop_sequence': ['S1', 'S2', 'S4', 'S5', 'S3', 'S1'],
44     'direction_morning': 'forward',
45     'direction_noon': 'forward',
46 },
47 ]
48 edge_vc = {}
49 edge_risk = {}
50 for _, row in lf.iterrows():
51     u, v = row['from'], row['to']
52     edge_vc[(u, v)] = float(row['saturation'])
53     edge_risk[(u, v)] = float(row['risk'])
54 BPR_ALPHA = 0.15
55 BPR_BETA = 4.0
56 BUS_SPEED_MPS = 5.56
57 def bpr_time_sec(length, vc):
58     t0 = length / BUS_SPEED_MPS
59     vc = max(float(vc), 0.0)
60     return t0 * (1 + BPR_ALPHA * (vc ** BPR_BETA))
61 for u, v, data in G.edges(data=True):
62     length = float(data.get('length', 50))
63     vc = edge_vc.get((u, v), 0.0)
64     data['bus_time'] = bpr_time_sec(length, vc)
65 def compute_route_path(G, stop_seq, forward=True):
66     seq = stop_seq if forward else list(reversed(stop_seq))
67     full_path = []
68     total_length = 0
69     total_time = 0
70     vc_list = []
71     risk_list = []
72     for k in range(len(seq) - 1):
73         s_from = seq[k]
74         s_to = seq[k + 1]
75         u = stop_to_node.get(s_from, '')
76         v = stop_to_node.get(s_to, '')
77         if u not in G or v not in G:
78             continue
79         try:
80             path = nx.shortest_path(G, u, v, weight='bus_time')
81         except nx.NetworkXNoPath:
82             continue
83         start_idx = 1 if full_path else 0
84         full_path.extend(path[start_idx:])

```

```

85     seg_length = 0
86     seg_time = 0
87     for i in range(len(path) - 1):
88         a, b = path[i], path[i+1]
89         if G.has_edge(a, b):
90             L = float(G[a][b].get('length', 50))
91             vc = edge_vc.get((a, b), 0)
92             seg_length += L
93             seg_time += bpr_time_sec(L, vc)
94             vc_list.append(vc)
95             risk_list.append(edge_risk.get((a, b), 0))
96         total_length += seg_length
97         total_time += seg_time
98     total_time_sec = total_time + DELTA * 60 * (len(seq) - 1)
99     total_time_min = total_time_sec / 60
100     return full_path, total_length, total_time_min, vc_list, risk_list
101 rows = []
102 eval_rows = []
103 for route in CANDIDATE_ROUTES:
104     rid = route['route_id']
105     rname = route['route_name']
106     rtype = route['route_type']
107     stop_seq = route['stop_sequence']
108     path, length, t_min, vcs, risks = compute_route_path(G, stop_seq, forward=True)
109     avg_vc = np.mean(vcs) if vcs else 0
110     avg_risk = np.mean(risks) if risks else 0
111     max_vc = max(vcs) if vcs else 0
112     path_str = ' → '.join(path[:12]) + ('...' if len(path) > 12 else '')
113     stop_str = ' → '.join([f"{stop_name.get(s,s)}" for s in stop_seq])
114     rows.append({
115         'route_id': rid,
116         'route_name': rname,
117         'route_type': rtype,
118         'stop_sequence': stop_str,
119         'path_nodes': path_str,
120         'num_nodes': len(path),
121         'length_m': f"{length:.0f}",
122         'travel_time_min': f"{t_min:.1f}",
123         'avg_vc': f"{avg_vc:.3f}",
124         'max_vc': f"{max_vc:.3f}",
125         'avg_risk': f"{avg_risk:.0f}",
126     })
127     print(f"\n{rid} {rname} ({rtype}):")
128     print(f"  站点: {stop_str}")
129     print(f"  长度: {length:.0f}m, 用时: {t_min:.1f}min (含停站)")
130     print(f"  经过节点: {len(path)} 个")
131     print(f"  平均 V/C: {avg_vc:.3f}, 最大 V/C: {max_vc:.3f}, 平均风险: {avg_risk:.0f}")
132 print(f"\n{'='*60}")
133 print("5. 广义成本评价:")
134 lengths = [float(r['length_m']) for r in rows]

```

```

135 times = [float(r['travel_time_min']) for r in rows]
136 vcs_avg = [float(r['avg_vc']) for r in rows]
137 risks_avg = [float(r['avg_risk']) for r in rows]
138 L_max = max(lengths) if max(lengths) > 0 else 1
139 T_max = max(times) if max(times) > 0 else 1
140 V_max = max(vcs_avg) if max(vcs_avg) > 0 else 1
141 S_max = max(risks_avg) if max(risks_avg) > 0 else 1
142 BUILDING_CAPACITY = {
143     'T_16': 3000, 'T_25': 2000,
144     'T_1': 1000, 'T_2': 1000, 'T_3': 1000, 'T_4': 1000, 'T_5': 1000,
145     'T_6': 1000, 'T_7': 1000, 'T_8': 1000, 'T_9': 1000, 'T_10': 1000,
146     'TYGY': 13000, 'XYGY': 7000, 'CYGY': 3000, 'GYGY': 5000,
147     'TYST': 1100, 'XYST': 1100, 'GYST': 1100, 'SYJ': 5000,
148 }
149 sel_stops_info = dict(zip(sel_df['stop_id'], sel_df['covered_demand']))
150 for i, route in enumerate(CANDIDATE_ROUTES):
151     unique_stops = set(route['stop_sequence'])
152     covered = sum(float(sel_stops_info.get(s, '0')) for s in unique_stops)
153     L = float(rows[i]['length_m'])
154     T = float(rows[i]['travel_time_min'])
155     V = float(rows[i]['avg_vc'])
156     S = float(rows[i]['avg_risk'])
157     P = covered
158     P_max = sum(BUILDING_CAPACITY.values())
159     G_cost = (ALPHA_L * L / L_max
160              + ALPHA_T * T / T_max
161              + ALPHA_V * V / V_max
162              + ALPHA_S * S / S_max
163              - ALPHA_P * P / P_max)
164     eval_rows.append({
165         'route_id': route['route_id'],
166         'route_name': route['route_name'],
167         'length_m': f"{L:.0f}",
168         'time_min': f"{T:.1f}",
169         'avg_vc': f"{V:.3f}",
170         'avg_risk': f"{S:.0f}",
171         'covered_demand': f"{P:.0f}",
172         'G_cost': f"{G_cost:.4f}",
173     })
174     print(f" {route['route_id']} {route['route_name']:10s}: "
175           f"L={L:.0f}m T={T:.1f}min V/C={V:.3f} S={S:.0f} "
176           f"覆盖={P:.0f} G={G_cost:.4f}")
177 best = min(eval_rows, key=lambda r: float(r['G_cost']))
178 print(f"\n 最优线路: {best['route_id']} {best['route_name']} (G={best['G_cost']})")
179 df_routes = pd.DataFrame(rows)
180 df_routes.to_csv(os.path.join(OUTPUT_DIR, "candidate_routes.csv"), index=False, encoding="utf-8-sig")
181 df_eval = pd.DataFrame(eval_rows)
182 df_eval.to_csv(os.path.join(OUTPUT_DIR, "route_evaluation.csv"), index=False, encoding="utf-8-sig")

```

```

183 print(f"\n输出: {OUTPUT_DIR}/")
184 print(f" candidate_routes.csv ({len(rows)} 条候选线路)")
185 print(f" route_evaluation.csv (线路评价)")

```

附录 Q: 步骤十六: 接驳需求识别与调度 (step16_bus_demand.py)

```

1 import pandas as pd
2 import numpy as np
3 import os
4 import math
5 STEP3_DIR = "output_step3"
6 STEP11_DIR = "output_step11"
7 STEP15_DIR = "output_step15"
8 OUTPUT_DIR = "output_step16"
9 os.makedirs(OUTPUT_DIR, exist_ok=True)
10 K = 25
11 HOURS = {
12     'morning': 45,
13     'noon': 50,
14     'evening': 40,
15 }
16 DELTA = 0.5
17 H_MAX = 10
18 M_MAX = 30
19 C_V = 8.0
20 C_W = 0.6
21 C_O = 0.2
22 PERIOD_LABEL = {'morning': '早高峰', 'noon': '午高峰', 'evening': '晚高峰'}
23 capacity_df = pd.read_csv(os.path.join(STEP11_DIR, "final_capacity.csv"))
24 N_max = sum(int(r['N_opt']) for _, r in capacity_df.iterrows())
25 print(f"校园最大容量 N_max = {N_max} 辆")
26 STEP2_DIR = "output_step2"
27 D_bike = {}
28 for p in ['morning', 'noon', 'evening']:
29     df = pd.read_csv(os.path.join(STEP2_DIR, f'od_{p}_b.csv'), index_col=0)
30     D_bike[p] = df.values.sum()
31 print(f"D_bike: {D_bike}")
32 ETA_BASE = 0.10
33 ETA_OVER = 0.50
34 D_over_raw = {p: max(0, D_bike[p] - N_max) for p in D_bike}
35 D_base = {p: ETA_BASE * D_bike[p] for p in D_bike}
36 D_over_bus = {p: ETA_OVER * D_over_raw[p] for p in D_bike}
37 print("\n各时段电动车需求与接驳需求:")
38 for p in ['morning', 'noon', 'evening']:
39     D_total = D_base[p] + D_over_bus[p]
40     print(f" {PERIOD_LABEL[p]:4s}: D_bike={D_bike[p]:.0f}, "
41           f"基础替代={D_base[p]:.0f}, 超载转移={D_over_bus[p]:.0f}, "
42           f"总接驳需求={D_total:.0f} 人次")
43 scenarios = {

```

```

44     '保守': 0.05,
45     '基准': 0.10,
46     '积极': 0.15,
47 }
48 demand_rows = []
49 for name, eta_base_sc in scenarios.items():
50     row = {'scenario': name, 'eta': f"{eta_base_sc:.0%}"}
51     for p in ['morning', 'noon', 'evening']:
52         d_bus = eta_base_sc * D_bike[p] + ETA_OVER * D_over_raw[p]
53         row[f'{p}_demand'] = f"{d_bus:.0f}"
54     demand_rows.append(row)
55 df_demand = pd.DataFrame(demand_rows)
56 df_demand.to_csv(os.path.join(OUTPUT_DIR, "bus_demand_scenarios.csv"),
57                 index=False, encoding="utf-8-sig")
58 print("\n接驳车需求（三情景 × 三时段）：")
59 for _, r in df_demand.iterrows():
60     print(f" {r['scenario']:4s} (={r['eta']}): "
61           f"早={r['morning_demand']} 午={r['noon_demand']} 晚={r['evening_demand']} 人次")
62 route_df = pd.read_csv(os.path.join(STEP15_DIR, "candidate_routes.csv"))
63 best_route = route_df[route_df['route_id'] == 'R1'].iloc[0]
64 tau_r = float(best_route['travel_time_min'])
65 print(f"\n最优线路: {best_route['route_name']}")
66 print(f" 单程时间  _r = {tau_r:.1f} min")
67 def evaluate_dispatch(D_s, H_t, tau_r, n):
68     headway = tau_r / n
69     capacity = n * K * (H_t / tau_r)
70     load_factor = D_s / max(capacity, 1)
71     avg_wait = headway / 2
72     operation_cost = C_V * n * H_t
73     waiting_cost = C_W * D_s * avg_wait
74     waste_cost = C_0 * max(0, capacity - D_s)
75     total_cost = operation_cost + waiting_cost + waste_cost
76     feasible = (capacity >= D_s) and (headway <= H_MAX)
77     return {
78         'n': n,
79         'headway': headway,
80         'capacity': capacity,
81         'load_factor': load_factor,
82         'avg_wait': avg_wait,
83         'operation_cost': operation_cost,
84         'waiting_cost': waiting_cost,
85         'waste_cost': waste_cost,
86         'total_cost': total_cost,
87         'feasible': feasible,
88     }
89 def optimize_dispatch(D_s, H_t, tau_r):
90     candidates = [evaluate_dispatch(D_s, H_t, tau_r, n) for n in range(1, M_MAX + 1)]
91     feasible = [c for c in candidates if c['feasible']]
92     if feasible:
93         best = min(feasible, key=lambda c: c['total_cost'])

```

```

94     best['constraint_note'] = '满足运能和发车间隔约束'
95 else:
96     capacity_feasible = [c for c in candidates if c['capacity'] >= D_s]
97     if capacity_feasible:
98         best = min(capacity_feasible, key=lambda c: c['total_cost'])
99         best['constraint_note'] = '满足运能, 但发车间隔超过约束'
100     else:
101         best = max(candidates, key=lambda c: c['capacity'])
102         best['constraint_note'] = '最大车辆数下仍运能不足'
103     return best, candidates
104 TIDAL_DIR = {
105     'morning': '宿舍区 → 教学区',
106     'noon': '教学区 → 食堂/商业街',
107     'evening': '教学区 → 宿舍/食堂',
108 }
109 BASE_ETA = 0.10
110 schedule_rows = []
111 cost_rows = []
112 for period in ['morning', 'noon', 'evening']:
113     D_s = BASE_ETA * D_bike[period] + ETA_OVER * D_over_raw[period]
114     H_t = HOURS[period]
115     best_dispatch, candidates = optimize_dispatch(D_s, H_t, tau_r)
116     n_t = best_dispatch['n']
117     h_t = best_dispatch['headway']
118     Q_cap = best_dispatch['capacity']
119     load_factor = best_dispatch['load_factor']
120     for c in candidates:
121         cost_rows.append({
122             'period': period,
123             'period_name': PERIOD_LABEL[period],
124             'n_vehicles': c['n'],
125             'headway_min': f"{c['headway']:.2f}",
126             'Q_cap': f"{c['capacity']:.1f}",
127             'load_factor': f"{c['load_factor']:.3f}",
128             'operation_cost': f"{c['operation_cost']:.2f}",
129             'waiting_cost': f"{c['waiting_cost']:.2f}",
130             'waste_cost': f"{c['waste_cost']:.2f}",
131             'total_cost': f"{c['total_cost']:.2f}",
132             'feasible': c['feasible'],
133         })
134     schedule_rows.append({
135         'period': period,
136         'period_name': PERIOD_LABEL[period],
137         'direction': TIDAL_DIR[period],
138         'D_s': f"{D_s:.0f}",
139         'H_t_min': H_t,
140         'tau_r_min': f"{tau_r:.1f}",
141         'n_vehicles': n_t,
142         'headway_min': f"{h_t:.1f}",
143         'avg_wait_min': f"{best_dispatch['avg_wait']:.1f}",

```



```

144     'Q_cap': f"{Q_cap:.0f}",
145     'load_factor': f"{load_factor:.2f}",
146     'operation_cost': f"{best_dispatch['operation_cost']:.2f}",
147     'waiting_cost': f"{best_dispatch['waiting_cost']:.2f}",
148     'waste_cost': f"{best_dispatch['waste_cost']:.2f}",
149     'total_cost': f"{best_dispatch['total_cost']:.2f}",
150     'meets_constraint': ' ' if best_dispatch['feasible'] else ' ',
151     'constraint_note': best_dispatch['constraint_note'],
152 })
153 print(f"\n {PERIOD_LABEL[period]}:")
154 print(f"  接驳需求 D_s = {D_s:.0f} 人次")
155 print(f"  最优车辆数 n = {n_t} 辆")
156 print(f"  发车间隔 h = {tau_r:.1f}/{n_t} = {h_t:.1f} min")
157 print(f"  平均等待 W = {best_dispatch['avg_wait']:.1f} min")
158 print(f"  运能 Q_cap = {n_t}×{K}×({H_t}/{tau_r:.1f}) = {Q_cap:.0f} 人次")
159 print(f"  满载率 = {D_s:.0f}/{Q_cap:.0f} = {load_factor:.2f}")
160 print(f"  成本 = 运营 {best_dispatch['operation_cost']:.1f} + 等待 {best_dispatch['waiting_cost']:.1f} + 浪费 {best_dispatch['waste_cost']:.1f} = {best_dispatch['total_cost']:.1f}")
161 print(f"  {best_dispatch['constraint_note']}")
162 df_sched = pd.DataFrame(schedule_rows)
163 df_sched.to_csv(os.path.join(OUTPUT_DIR, "bus_schedule.csv"), index=False, encoding="utf-8-sig")
164 df_cost = pd.DataFrame(cost_rows)
165 df_cost.to_csv(os.path.join(OUTPUT_DIR, "dispatch_cost_candidates.csv"), index=False, encoding="utf-8-sig")
166 print(f"\n{' '*60}")
167 print(f"接驳车系统设计汇总（基准情景 = {BASE_ETA:.0%}）:")
168 total_vehicles = sum(int(r['n_vehicles']) for r in schedule_rows)
169 max_vehicles = max(int(r['n_vehicles']) for r in schedule_rows)
170 print(f"  线路: {best_route['route_name']} (单程 {tau_r:.1f} min)")
171 print(f"  三时段合计需要 {total_vehicles} 车次")
172 print(f"  最大同时投放: {max_vehicles} 辆 ({PERIOD_LABEL[max(schedule_rows, key=lambda r: int(r['n_vehicles'])]['period'])})")
173 print(f"\n输出: {OUTPUT_DIR}/")
174 print(f"  bus_demand_scenarios.csv")
175 print(f"  bus_schedule.csv")
176 print(f"  dispatch_cost_candidates.csv")

```

附录 R：步骤十七：微公交效果评价 (step17_effect.py)

```

1 import pandas as pd
2 import numpy as np
3 import networkx as nx
4 import os
5 STEP1_DIR = "output_step1"
6 STEP2_DIR = "output_step2"
7 STEP3_DIR = "output_step3"
8 STEP8_DIR = "output_step8"

```

```

9 STEP11_DIR = "output_step11"
10 STEP16_DIR = "output_step16"
11 OUTPUT_DIR = "output_step17"
12 os.makedirs(OUTPUT_DIR, exist_ok=True)
13 MODES = ['p', 'b', 'c', 'bus']
14 ETA = 0.10
15 BUS_PCU = 1.5
16 PERIODS = ['morning', 'noon', 'evening']
17 PERIOD_LABEL = {'morning': '早高峰', 'noon': '午高峰', 'evening': '晚高峰'}
18 capacity_df = pd.read_csv(os.path.join(STEP11_DIR, "final_capacity.csv"))
19 N_max = sum(int(r['N_opt']) for _, r in capacity_df.iterrows())
20 zone_map = pd.read_csv(os.path.join(STEP8_DIR, "building_zone_map.csv"))
21 zones_df = pd.read_csv(os.path.join(STEP8_DIR, "zones.csv"))
22 z2name = dict(zip(zones_df['zone_id'], zones_df['zone_name']))
23 with open(os.path.join(STEP3_DIR, "baseline.txt"), "r") as f:
24     baseline_lines = f.readlines()
25 baseline = {}
26 for line in baseline_lines[1:]:
27     period, ttt, s_val = line.strip().split(",")
28     baseline[period] = {'TTT': float(ttt), 'S': float(s_val)}
29 schedule_df = pd.read_csv(os.path.join(STEP16_DIR, "bus_schedule.csv"))
30 D_bike_old = {'morning': 4146, 'noon': 4393, 'evening': 5323}
31 D_bus = {}
32 for _, row in schedule_df.iterrows():
33     D_bus[row['period']] = float(row['D_s'])
34 D_bike_new = {}
35 for p in PERIODS:
36     D_bike_new[p] = D_bike_old[p] - D_bus.get(p, 0)
37     D_bike_new[p] = max(D_bike_new[p], 0)
38 theta_old = {p: D_bike_old[p] / N_max for p in PERIODS}
39 theta_new = {p: D_bike_new[p] / N_max for p in PERIODS}
40 print("电动车需求与饱和度变化:")
41 print(f" {'时段':<6} {'原需求':>6} {'接驳替代':>8} {'新需求':>6} {'原 ':>7} {'新 ':>7} {'改善':>8}")
42 for p in PERIODS:
43     delta_theta = (theta_old[p] - theta_new[p]) / max(theta_old[p], 1e-9) * 100
44     print(f" {PERIOD_LABEL[p]:<6} {D_bike_old[p]:>6} {D_bus.get(p,0):>8.0f} "
45           f"{D_bike_new[p]:>6.0f} {theta_old[p]:>7.3f} {theta_new[p]:>7.3f} {delta_theta:>7.1f}%")
46 print("\n区域压力变化（早高峰为主要改善时段）:")
47 print(f" {'区域':<12} {'停车容量':>8} {'原 ':>7} {'改善后 ':>10} {'状态':<10}")
48 bus_reduction = D_bus.get('morning', 0)
49 zone_rho_old = {}
50 for _, row in capacity_df.iterrows():
51     zone_rho_old[row['zone_name']] = float(row['rho'])
52 BUILDING_CAP = {
53     'T_16': 3000, 'T_25': 2000,
54     'T_1': 1000, 'T_2': 1000, 'T_3': 1000, 'T_4': 1000, 'T_5': 1000,
55     'T_6': 1000, 'T_7': 1000, 'T_8': 1000, 'T_9': 1000, 'T_10': 1000,
56     'TYGY': 13000, 'XYGY': 7000, 'CYGY': 3000, 'GYGY': 5000,

```

```

57     'TYST': 1100, 'XYST': 1100, 'GYST': 1100, 'SYJ': 5000,
58 }
59 zone_caps = {}
60 for _, bz_row in zone_map.iterrows():
61     bld = bz_row['building']
62     zid = bz_row['zone_id']
63     zname = z2name.get(zid, '')
64     if bld in BUILDING_CAP:
65         zone_caps[zname] = zone_caps.get(zname, 0) + BUILDING_CAP[bld]
66 total_cap = sum(zone_caps.values())
67 effect_rows = []
68 for _, row in capacity_df.iterrows():
69     zname = row['zone_name']
70     N_opt = int(row['N_opt'])
71     D_old = float(row['D_i'])
72     old_rho = float(row['rho'])
73     z_share = zone_caps.get(zname, 1000) / total_cap
74     reduction = bus_reduction * z_share
75     D_new = max(0, D_old - reduction)
76     new_rho = D_new / N_opt if N_opt > 0 else 0
77     if new_rho < 0.85:
78         status = '容量充足'
79     elif new_rho <= 1.00:
80         status = '接近饱和'
81     else:
82         status = '超载'
83     delta = (old_rho - new_rho) / max(old_rho, 1e-9) * 100
84     print(f" {zname:<12} {N_opt:>8} {old_rho:>7.3f} {new_rho:>10.3f} {status:<10} (-{delta:.1f}
85           }%)")
86     effect_rows.append({
87         'zone_name': zname,
88         'N_opt': N_opt,
89         'D_old': f"{D_old:.0f}",
90         'D_new': f"{D_new:.0f}",
91         'rho_old': f"{old_rho:.3f}",
92         'rho_new': f"{new_rho:.3f}",
93         'delta_rho': f"{delta:.1f}%",
94         'status_old': row['status'],
95         'status_new': status,
96     })
97 print("\n重新运行 UE 评估接驳车对路网的影响（每时段独立重算）:")
98 import sys, math
99 _built = False
100 try:
101     _step4_src = open(os.path.join(os.path.dirname(__file__), "step4_scenarios.py")).read().
102         split("#          情景方案")[0]
103     _saved_output_dir = OUTPUT_DIR
104     exec(compile(_step4_src, "step4_scenarios.py", 'exec'))
105     OUTPUT_DIR = _saved_output_dir
106     _built = True

```

```

105 except Exception as _e:
106     print(f" (无法加载 step4, 回退比例估算: {_e})")
107 ttt_rows = []
108 for p in PERIODS:
109     ttt_old = baseline[p]['TTT']
110     s_old = baseline[p]['S']
111     if D_bike_old[p] == 0:
112         continue
113     ebike_red = D_bus.get(p, 0) / max(D_bike_old[p], 1)
114     if _built and ebike_red > 0:
115         try:
116             if 'G' not in dir():
117                 G = nx.read_gml(os.path.join(STEP1_DIR, "campus_graph.gml"))
118             od_new = {}
119             for mode in MODES:
120                 fp = os.path.join(STEP2_DIR, f"od_{p}_{mode}.csv")
121                 if not os.path.exists(fp): continue
122                 df = pd.read_csv(fp, index_col=0)
123                 for src in df.index:
124                     for dst in df.columns:
125                         v = float(df.loc[src, dst])
126                         if v < 0.01: continue
127                         if mode == 'b':
128                             v *= (1 - ebike_red)
129                         if v > 0.01:
130                             od_new[(src, dst, mode)] = v
131             adj, eid_map, eattr = build_adj(G)
132             x_mode, x_eq = frank_wolfe(od_new, adj, eattr, eid_map, max_iter=20)
133             ttt_new = calc_ttt(x_mode, x_eq, eattr)
134             s_new, _ = calc_conflict(x_mode, eattr)
135         except Exception:
136             ttt_new = ttt_old * (1 - ebike_red * 0.25)
137             s_new = s_old * (1 - ebike_red * 0.35)
138     else:
139         ttt_new = ttt_old * (1 - ebike_red * 0.25)
140         s_new = s_old * (1 - ebike_red * 0.35)
141     delta_ttt = (ttt_old - ttt_new) / max(ttt_old, 1) * 100
142     delta_s = (s_old - s_new) / max(s_old, 1) * 100
143     ttt_rows.append({
144         'period': p, 'period_name': PERIOD_LABEL[p],
145         'TTT_old': f"{ttt_old:.0f}", 'TTT_new': f"{ttt_new:.0f}",
146         'delta_TTT': f"{delta_ttt:.1f}%", 'S_old': f"{s_old:.0f}",
147         'S_new': f"{s_new:.0f}", 'delta_S': f"{delta_s:.1f}%",
148         'ebike_reduction': f"{ebike_red*100:.1f}%",
149     })
150     print(f" {PERIOD_LABEL[p]}: e-bike -{ebike_red*100:.1f}% "
151           f"→ TTT {ttt_old:.0f}→{ttt_new:.0f} ({delta_ttt:+.1f}%) "
152           f"S {s_old:.0f}→{s_new:.0f} ({delta_s:+.1f}%)")
153 df_zone = pd.DataFrame(effect_rows)
154 df_zone.to_csv(os.path.join(OUTPUT_DIR, "bus_effect_evaluation.csv"),

```

```

155         index=False, encoding="utf-8-sig")
156 df_ttt = pd.DataFrame(ttt_rows)
157 df_ttt.to_csv(os.path.join(OUTPUT_DIR, "bus_effect_ttt.csv"),
158               index=False, encoding="utf-8-sig")
159 worst_p = max(PERIODS, key=lambda p: D_bike_old.get(p, 0))
160 report = os.path.join(OUTPUT_DIR, "bus_effect_report.txt")
161 with open(report, "w", encoding="utf-8") as f:
162     f.write("=" * 70 + "\n")
163     f.write("浙师大校园微公交系统效果评价报告\n")
164     f.write("=" * 70 + "\n\n")
165     f.write("一、接驳需求\n")
166     f.write(f"  校园最大电动车容量 N_max = {N_max} 辆\n")
167     f.write(f"  早高峰电动车峰值需求 = {D_bike_old['morning']} 辆\n")
168     f.write(f"  接驳需求模型: 基础替代(_base={ETA:.0%}) + 超载转移(_over=50%)\n")
169     f.write(f"  基准情景: 微公交接驳需求 = {D_bus.get('morning',0):.0f} 人次\n\n")
170     f.write("二、改善效果\n")
171     f.write(f"  早高峰电动车需求: {D_bike_old['morning']} → {D_bike_new['morning']:.0f}\n")
172     f.write(f"  校园电动车饱和度: {theta_old['morning']:.3f} → {theta_new['morning']:.3f}\n")
173     delta_th = (theta_old['morning'] - theta_new['morning']) / theta_old['morning'] * 100
174     f.write(f"  饱和度改善: {delta_th:.1f}%\n\n")
175     f.write("三、典型区域压力变化\n")
176     for r in effect_rows:
177         f.write(f"  {r['zone_name']}: {r['rho_old']} → {r['rho_new']} ({r['delta_rho']})\n")
178     f.write(f"\n四、路网效果（早高峰）\n")
179     if ttt_rows:
180         r = ttt_rows[0]
181         f.write(f"  TTT: {r['TTT_old']} → {r['TTT_new']} ({r['delta_TTT']})\n")
182         f.write(f"  冲突风险 S: {r['S_old']} → {r['S_new']} ({r['delta_S']})\n")
183     f.write(f"\n五、结论\n")
184     f.write(f"  微公交系统可将早高峰电动车饱和度从 {theta_old['morning']:.3f} "
185           f"降至 {theta_new['morning']:.3f}, 改善约 {delta_th:.1f}%.\n")
186     f.write(f"  作为问题一综合路网优化和问题二总量控制政策的重要补充, \n")
187     f.write(f"  微公交系统是实现校园交通可持续发展的关键措施.\n")
188 print(f"\n输出: {OUTPUT_DIR}/")
189 print(f" bus_effect_evaluation.csv")
190 print(f" bus_effect_report.txt")

```